



Clustering-based return prediction model for stock pre-selection in portfolio optimization using PSO-CNN+MVF

Mahdi Ashrafzadeh^a, Hasan Mehtari Taheri^a, Mahmoud Gharehgozlou^a, Sarfaraz Hashemkhani Zolfani^{b,*}

^a Department of Industrial Engineering and Management Systems, Amirkabir University of Technology, Tehran, Iran

^b School of Engineering, Universidad Catolica del Norte, Larrondo 1281, Coquimbo, Chile

ARTICLE INFO

Article history:

Received 14 May 2023

Revised 19 July 2023

Accepted 25 August 2023

Available online 29 August 2023

Keywords:

Stock pre-selection

K-means

Convolutional neural networks

Particle Swarm Optimization (PSO)

Portfolio optimization

ABSTRACT

Incorporating return prediction in portfolio optimization can make portfolio optimization more efficient by selecting the stocks expected to perform well in the future. This paper proposes a hybrid method that integrates a convolutional neural network (CNN) with optimized hyperparameters by the particle swarm optimization (PSO) for stock pre-selection and a mean-variance with forecasting (MVF) model for portfolio optimization. In the stock pre-selection step, to reduce the computational complexity of the model, the CNN network is trained on the clustered stocks via the K-means method instead of training on each stock. The proposed model also includes a novel feature selection method that weighs features based on their impact on predicting stock returns for more accurate predictions. The results of implementing this model on 21 stocks from the New York Stock Exchange (NYSE) market demonstrate that the proposed method for training the CNN network on clustered stocks does not significantly differ in prediction accuracy to conventional methods. Moreover, in the portfolio optimization step, the returns predicted in the stock pre-selection step are used to optimize the weight of stocks in the portfolio. Compared to other benchmark models, the proposed model exhibits superior financial performance.

© 2023 The Author(s). Published by Elsevier B.V. on behalf of King Saud University. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

1. Introduction

The effective management of financial assets is crucial for investors seeking to generate positive returns while minimizing risks. Portfolio optimization has become a popular approach for investors, with the mean-variance (MV) model proposed by Markowitz (1952) being a successful example. Despite advancements in portfolio optimization (Wang and Aste, 2023; Zolfani et al., 2022; Gupta, 2022), incorporating machine learning (ML) models can further improve performance. By utilizing predictive models to select assets and predicted returns during the optimization process, investors can enhance portfolio performance. Incorporating advanced methods offers an opportunity to achieve effective management of financial assets.

The pre-selection of assets is a critical step in portfolio management as it can impact a portfolio's overall performance and risk. Selecting the right assets can be challenging, and failure to do so

can lead to suboptimal portfolios that do not meet investment objectives (Wang et al., 2020). Return prediction is one way to pre-select assets in portfolio optimization, and statistical methods such as time-series and regression analyses have been used for this purpose (Ma et al., 2020; Bini and Mathew, 2016; Adebisi et al., 2014; Enke et al., 2011). However, the complexity and non-linearity of financial time series data pose challenges to these methods (Alhnaity and Abbod, 2020). To address this, a wide range of machine learning (ML) models have been developed to deal with the complex, multidimensional, and noisy time series data in recent years (Bhandari et al., 2022; Henrique et al., 2018). These ML models offer new avenues for effective pre-selection of assets in portfolio optimization, particularly in dealing with the instability of the stock market and correlation between investment psychology and market behavior.

Based on the efficient market theory (Jensen, 1978), financial markets cannot be predicted. However, some studies have achieved satisfactory accuracy in predicting financial time series by taking advantage of some inefficiency in all financial markets. There are two main categories of financial time series prediction models: 1- Statistical models, which are divided into two groups. The first group is average-oriented models such as the auto-

* Corresponding author.

E-mail addresses: mahdi.ashrafzade@aut.ac.ir (M. Ashrafzadeh), hasan.m.t@aut.ac.ir (H.M. Taheri), gharehgozlou@aut.ac.ir (M. Gharehgozlou), sa.hashemkhani@gmail.com (S. Hashemkhani Zolfani).

regressive integrated moving average (ARIMA) (Box and Jenkins, 1976), while the second group is based on fluctuations or variance, such as the autoregressive conditional heteroscedasticity (ARCH) (Engle, 1982), and the generalized autoregressive conditional heteroscedasticity (GARCH) (Bollerslev, 1986). 2- There are also some the ML models, such as the support vector regression (SVR) (Rasel et al., 2015), the random forest (RF) (Ballings et al., 2015), the long-short term memory (LSTM) (Roondiwala et al., 2017), the convolutional neural networks (CNN) (Hoseinzade and Haratizadeh, 2019), and ensemble learning methods such as the gradient boosted decision tree (GBDT) (Zhou et al., 2019), the adaptive boosting (AdaBoost) (Zhang et al., 2016), and the extreme gradient boosting (XGBoost) (Nobre and Neves, 2019). Among deep learning (DL) models, the CNNs consider the complexity of financial markets better than other models due to considering the relationship between input features through convolutional kernels, and their potential performance in predicting financial time series has been confirmed in many studies (Gunduz et al., 2017; Sezer and Ozbayoglu, 2018).

Although choosing the type of predictive model is crucial, different methods, such as optimizing *meta*-parameters of different models, how to train models, and using feature selection methods, play an important role in improving the performance of models. ML models are becoming increasingly sophisticated, with many parameters that have an enormous impact on performance, referred to as hyperparameters. To achieve optimal performance of these models, researchers have proposed several *meta*-heuristic algorithms designed to search for the best set of hyperparameters. These algorithms include the firefly algorithm (FA) (Nayak et al., 2021), the salp swarm algorithm (SSA) (Li et al., 2021), the gravitational search algorithm (GSA) (Ezzat et al., 2021), the artificial fish swarm algorithm (AFSA) (Hua et al., 2021), the genetic algorithm (GA) (Wicaksono and Supianto, 2018), and the particle swarm optimization (PSO) (Wang et al., 2019). One such algorithm, the FA, was improved by Chen et al. (2021) to avoid falling into local optima too soon, achieving better performance than other *meta*-heuristic algorithms. For instance, the improved FA demonstrated a stronger performance when optimizing the hyperparameters of XGBoost, which is a popular ensemble learning algorithm. Wicaksono and Supianto (2018) utilized GAs to optimize hyperparameters, highlighting the versatility of *meta*-heuristic algorithms as a tool for hyperparameter tuning. These approaches are highly flexible and can optimize different parameters, including regularization coefficients, learning rates, the number of hidden layers, and layer sizes, among others.

Another way to improve the performance of learning models is to use feature selection methods to select the appropriate number of input features. Feature selection effectively removes irrelevant and redundant data, reduces computation time, improves learning accuracy, and facilitates a better understanding of the learning model or data (Cai et al., 2018; Jianyu and Lingfeng, 2016). Moreover, the method of training the model significantly reduces the computational complexity of the model. In most studies, individual stock data have been used to train the models. This way of training dramatically increases the complexity of the model in portfolio optimization problems where a large number of stocks are evaluated. Therefore, to overcome this problem, before training the models on individual stocks, they can be clustered using stock clustering methods, and the training of the models is done on the representative of each cluster.

This paper aims to propose an investment decision-making model for individual investors that combines the PSO-CNN model, which concentrates on relationships between input features in modeling the complexity of financial markets in return prediction, and the MV model to portfolio construction. First, the CNN forecasts daily stock returns, and the model's hyperparameters are

optimized using the PSO. To reduce computational complexity, stocks are clustered using the K-means method, and the CNN training is performed on each cluster's representative. A feature selection method based on the CNN-LSTM model is proposed to improve model performance. The stocks with the highest predicted return are then selected to construct the portfolio. Finally, the MV model determines the optimal weight of selected stocks utilizing the predicted return from the first step as the expected return during optimization.

The main contributions of this study can be summarized as follows:

- Developing a return prediction model using the PSO-CNN for stock pre-selection and portfolio optimization.
- Developing a new method for training the DL models utilizing a cluster of stocks, which reduces computational complexity without compromising prediction accuracy. This training method no longer focuses on individual stocks.
- Developing a new feature selection method based on the CNN-LSTM, which can weigh the features based on their influence on the prediction of the target variable.

The following sections of the paper are organized thus: [Section 2](#) reviews studies on various DL models utilized in stock portfolio optimization to emphasize the research gap. [Section 3](#) describes our proposed mode. The results of the proposed and comparative models are presented in [Section 4](#). [Section 5](#) concludes the paper with conclusions and directions for future studies.

2. Literature review

Numerous investigations have centered around ameliorating the effectiveness of the MV model for optimizing portfolios. As ML models have progressed, a few studies have explored their fusion with the MV models. Paiva et al. (2019) proposed a model using a support vector machine (SVM) classifier and MV model for portfolio selection for day trading investments. The model was evaluated using the São Paulo Stock Exchange Index (Ibovespa), outperforming other benchmark models in terms of returns and risks. The study also evaluated the effect of brokerage costs on buying and selling stocks in the Brazilian market and the classifier's performance in portfolio optimization. Wang et al. (2020) utilized the LSTM and the MV models to construct a portfolio that captures long-term dependencies of financial time-series data and selects assets with higher potential returns. The results of their model on the UK Stock Exchange 100 Index (FTSE 100) between March 1994 and March 2019 illustrate that the proposed LSTM + MV model has superior performance in cumulative returns, Sharpe ratio, and average return to risk compared to other benchmark models. Chen et al. (2021) investigated the integration of forecasting results of the hybrid extreme gradient boosting (XGBoost) model, whose parameters are optimized with the improved firefly algorithm (IFA) with the MV model in the Shanghai stock market. The results reveal the superiority of their model in terms of return and risk compared to the benchmark models. Chaweewanchon and Chaysiri (2022) proposed a new portfolio construction model incorporating CNN and bidirectional LSTM to predict stock prices and optimize the portfolio. The Stock Exchange of Thailand 50 Index (SET50) results indicate that the proposed model outperforms the comparison models regarding risk, return, and Sharpe ratio. Ta et al. (2022) proposed using the LSTM to predict stock movements and construct an efficient portfolio. Portfolio optimization models were used to investigate performance, including equal-weighted modeling and optimization modeling the MV optimization. The results illustrated that the LSTM prediction

model had high accuracy and outperformed other prediction models. They confirmed that combining the LSTM with the MV model is suitable for portfolio construction.

Aside from utilizing ML models for portfolio optimization, another body of research has been dedicated to enhancing the MV model. Freitas et al. (2009) proposed a new portfolio optimization model that utilizes neural network predictors to capture short-term investment opportunities. The model derives a risk measure based on the prediction errors and selects predictors with low and complementary pairwise error profiles to enable efficient diversification. The evaluation of the model using real data from the Brazilian stock market showed that it outperforms the MV model and market index by taking advantage of short-term opportunities and generating normal prediction errors despite the non-normality of stock return time series. Ma et al. (2021) employed five different predictive models: the RF, SVR, LSTM, deep multi-layer perceptron (DMLP), and CNN. These models were used to pre-select stocks for portfolio optimization, and the predictive results were incorporated into an MV model with forecasting (MVF). The research analyzed the historical data of China Securities 100 Index (CSI 100) component stocks from 2007 to 2015. The study concluded that the RF + MVF model was the most suitable for daily investment trading. Chen et al. (2022) put forward a diversified portfolio selection approach, which is based on stock prediction and aims to improve the performance of portfolio models. The proposed model consists of two steps. The first step utilizes the ML models like SVM, LSTM, RF, extreme learning machine (ELM), and BPNN to select diversified stocks with high predicted returns. In the second step, predictions are incorporated into the Modified mean–variance (MMV) model to determine the asset allocation of each selection. The study used the component stocks of CSI 100 as a sample to test the proposed approach. The empirical results indicate that the RF + MMV model outperforms similar models and market indexes regarding return and risk measures. The model proposed by Behera et al. (2023) uses a hybrid model to select stocks with the greatest predicted returns using the ML models such as the RF, the adaptive boosting (AdaBoost), the

SVR, the k-nearest neighbors (KNN), and the artificial neural network (ANN). The next step employed the mean–VaR portfolio optimization model for portfolio selection. The results utilizing monthly data from three stock exchange operations showed that the mean–VaR model outperforms other benchmark models when coupled with AdaBoost prediction. Ma et al. (2023) employed an autoencoder (AE) to extract features and the LSTM model to predict stock returns. These were then incorporated into a portfolio optimization model alongside a worst-case omega model. In comparison to other models, including equally weighted portfolios and prediction-based portfolio optimization models, the proposed model surpassed them all. Additionally, even after accounting for transaction costs, the proposed model continues to outperform, indicating that it is an appropriate option for individuals who seek high returns in practice investments with considerable risk. Many researchers have expressed interest in combining the results of the ML models with portfolio optimization, as demonstrated by the studies covered in this section. This trend is expected to continue with the advancement of newer ML models. In this regard and following the above studies, this paper proposes a two-stage portfolio optimization model. The first step makes use of the PSO-CNN model to predict returns and pre-select stocks for portfolio construction. In the second step, the MVF model is used to determine the optimal weight of the chosen stocks. To improve the model's accuracy in predicting stock returns, we developed a new feature selection method based on the CNN-LSTM. Moreover, a clustering method is used to train the ML models, reducing computational complexity. Details of the studies discussed in this section are outlined in Table 1 for ease of understanding.

3. Proposed model for portfolio optimization

In this section, our proposed model for portfolio optimization is explained in detail in four sub-sections: data gathering and preprocessing, return prediction methodology, portfolio optimization methodology, and benchmark models and evaluation measures. The framework of the proposed model is illustrated in Fig. 1.

Table 1
Comparison of this study with existing studies in the literature.

Authors (year)	Prediction model	Portfolio optimization model	Feature selection	Hyperparameter optimization	clustering	Transaction cost	Dataset
Freitas et al. (2009)	Autoregressive moving referenceneural network(AR-MRNN)	Minimizing: the prediction-based portfolio risk	–	–	–	–	Brazilian stock market
Paiva et al. (2019)	SVM	MV	–	–	–	+	Ibovespa
Wang et al. (2020)	LSTM	MVF	+	–	–	+	FTSE 100 Index
Chen et al. (2021)	XGBoost	MV	–	+	–	–	Shanghai stock market
Ma et al. (2021)	RF, SVR, LSTM, DMLP, and CNN	MVF	–	–	–	–	CSI 100 Index
Ta et al. (2022)	LSTM	MV	–	–	–	–	S&P 500
Authors (year)	Prediction model	Portfolio optimization model	Feature selection	Hyperparameter optimization	clustering	Transaction cost	Dataset
Chaweewanchon and Chaysiri (2022)	CNN and BiLSTM	MV	–	–	–	–	SET50
Chen et al. (2022)	SVM, LSTM, RF, ELM, and BPNN	MMV	–	–	–	–	CSI 100 Index
Behera et al. (2023)	AdaBoost, SVR, KNN, and ANN	mean–VaR	–	–	–	–	Bombay Stock Exchange, Tokyo Stock Exchange, and Shanghai Stock Exchange
Ma et al. (2023)	LSTM	worst-case omega	+	–	–	+	CSI 100
This study	PSO-CNN, LSTM, and CNN-LSTM	MVF	+	+	+	+	21 stocks in NYSE

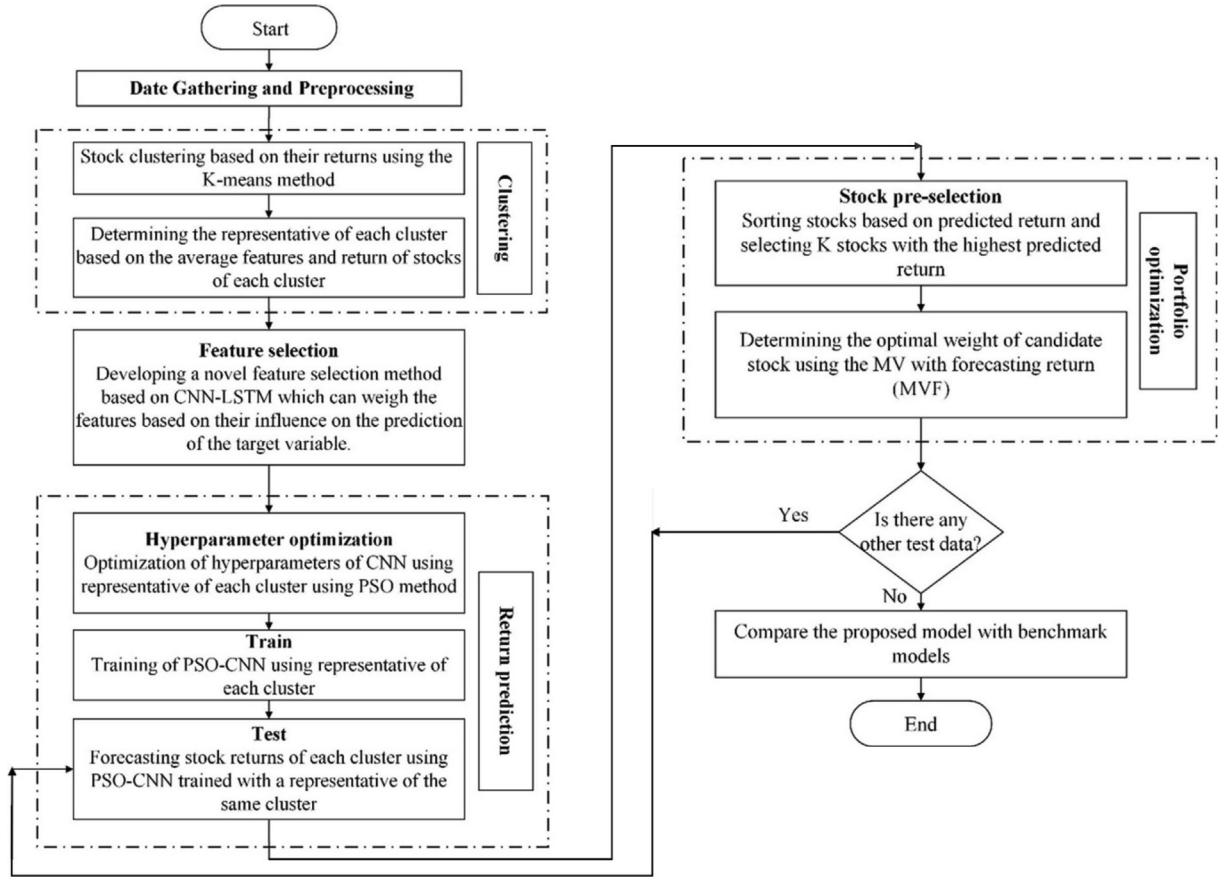


Fig. 1. The framework of the proposed model.

3.1. Data gathering and preprocessing

The performance of the proposed model and the benchmark models are evaluated on 21 randomly selected stocks from the NYSE market. We utilize daily data spanning ten years from 2012 to 2021. The training dataset consists of data from 2012 to 2018, while the validation dataset includes data from 2019. This data is used to adjust the optimal features and hyperparameters of the models. Data from 2020 and 2021 are utilized as test data for evaluating the models' performance. Table 2 illustrates the statistical characteristics of the input data.

Advanced models of multivariable time series prediction are used from input features calculated from stock prices in the form of indicators. These indicators can potentially provide more comprehensive insights into stock market data. This paper uses ten technical indicators as input features of models. The exponential moving average (EMA), the simple moving average (SMA), the moving average convergence/divergence (MACD), the percentage price oscillator (PPO), the price rate of change (ROC), the stochastic oscillator (Stochastic), the relative strength index (RSI), the average true range (ATR), the parabolic SAR (PSAR) and the stochastic RSI (Stochastic RSI) are technical indicators which are considered. As financial time-series prediction is always explained by lagged observations (Box et al., 2015), four lagged returns are considered input features. Appendix A describes selected features in more detail. To ensure that gradient descent in the DL models moves towards the minimum seamlessly and that the steps applied for gradient descent are updated at the same pace for all features, it is necessary to perform data scaling by implementing the min-max scaler equation (Eq. (1)) before feeding it into the models.

$$X_{\text{scaled}}^i = \frac{(X^i - X_{\min}^i)}{(X_{\max}^i - X_{\min}^i)} \quad i = 1, \dots, m \quad (1)$$

where X_{scaled}^i is the scaled value of feature i , $X_{\min}^i$ is the minimum value of feature i and $X_{\max}^i$ is the maximum value of feature i .

Feature selection methods will be used to select the features with the greatest impact on the target variable, and other features will be removed. The feature selection methods are implemented only for technical features, and the features based on return lag will be fixed inputs of the DL models.

3.2. Return prediction methodology

This section delineates the detail of the return prediction methodology incorporated in our proposed model. The methodology comprises four sub-sections: stock clustering, feature selection, hyperparameter optimization, and predictive model.

3.2.1. Stock clustering method

A clustering approach is utilized to reduce the computational complexity of both feature selection and model training for individual stocks. This involves grouping similar stocks together. Subsequently, the feature selection and prediction model training is carried out on a representative stock from each cluster instead of on each stock.

The K-means method is a well-known type of partitioning clustering algorithm, originally introduced by Forgy (1965) and

Table 2
Descriptive statistics for sample data.

Symbol	Mean	Std	Min	25%	50%	75%	Max
TSLA	61.91	96.77	1.52	13.36	17.07	28.46	409.97
GOOGL	54.26	35.03	13.99	27.73	46.20	64.66	149.84
AAPL	53.90	46.60	12.06	21.67	33.91	65.34	180.96
BA	181.59	96.60	55.67	111.95	145.98	239.85	430.30
MSFT	104.64	88.87	21.47	37.06	63.32	148.80	339.92
DIS	102.37	36.95	34.30	81.00	101.09	119.30	201.91
SONY	46.27	30.61	9.63	20.82	34.65	62.02	128.59
JPM	78.30	37.43	23.12	46.48	74.66	100.69	166.52
AOS	39.78	18.03	8.74	22.98	42.43	53.66	84.27
WMT	85.96	31.19	45.30	61.74	71.35	112.80	158.60
NVDA	56.15	71.86	2.61	4.53	28.57	64.74	333.49
AMZN	66.91	54.28	8.80	17.53	46.90	96.87	186.57
FDX	167.32	56.99	75.63	126.94	157.14	214.77	308.64
Symbol	Mean	Std	Min	25%	50%	75%	Max
NFLX	216.97	181.02	7.69	60.72	150.17	357.08	691.69
AMD	29.43	37.22	1.62	3.70	11.13	42.73	161.91
F	10.03	2.70	3.86	8.50	9.76	11.11	24.39
AAL	28.93	13.14	4.74	16.42	30.46	39.97	56.99
INTC	34.51	12.99	14.36	23.80	30.96	46.01	64.38
MCD	136.62	60.36	63.69	77.89	124.73	186.49	265.37
SEE	37.81	12.76	11.14	29.53	40.01	43.43	69.68

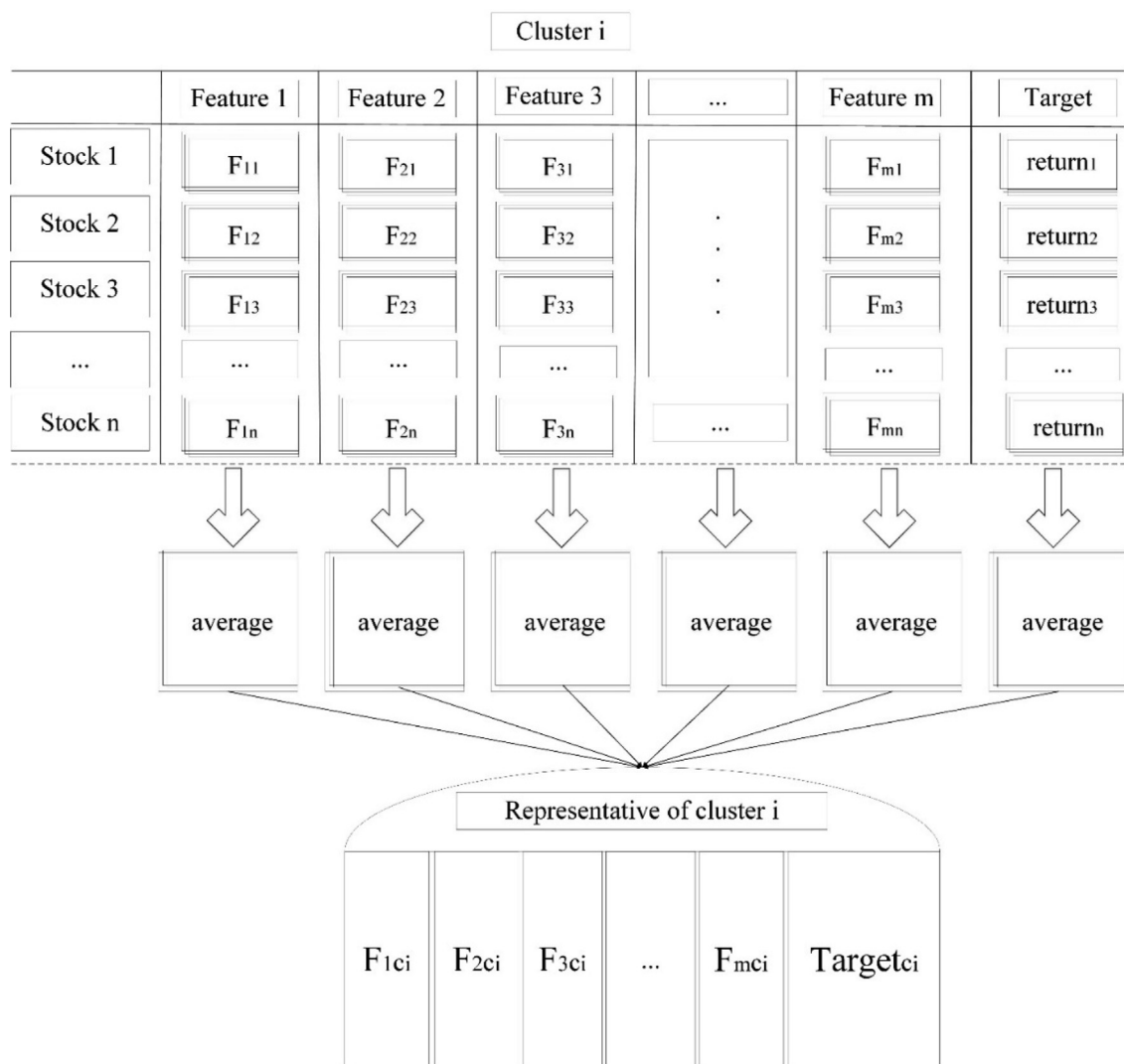


Fig. 2. The process of calculating representative of cluster.

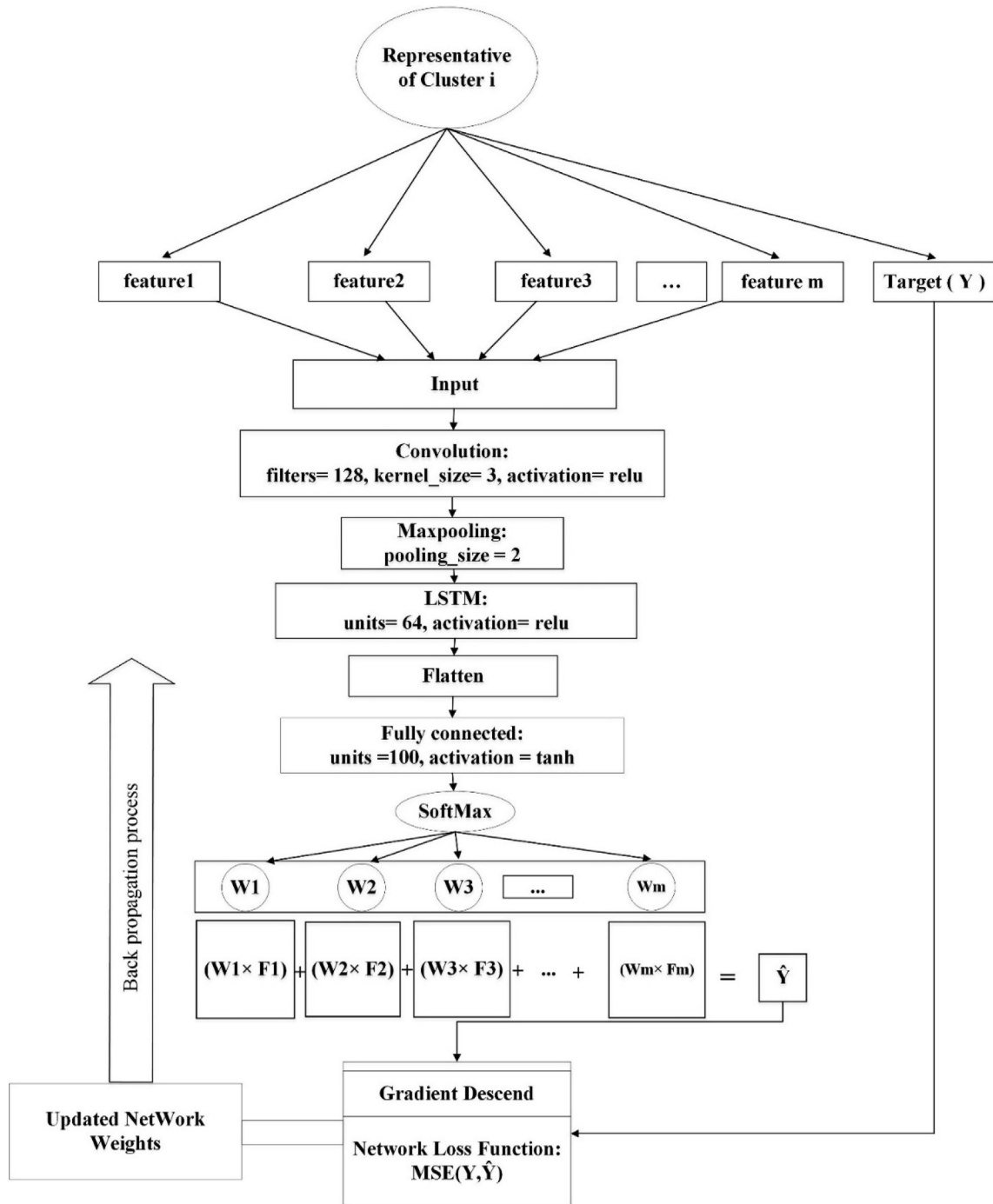


Fig. 3. Our proposed feature selection method.

MacQueen (1967), and a more widely used version of the algorithm published by Lloyd (1982). The K-means clustering aims to group N observations into k clusters, where each observation belongs to the cluster with the mean closest to its value. The K-means initiates with a single cluster whose center is the mean of the data. In our proposed model, stocks are categorized into various clusters, starting with the identification of the optimal number of clusters using the elbow method (Kodinariya and Makwana, 2013). This approach determines the trade-off point between the number of clusters and the inertia calculated using Eq. (2).

$$\sum_{i=1}^n (x_i - c_k)^2 \quad (2)$$

This method calculates the sum of squared distances between each sample's position, denoted as x_i , and the central point of the cluster, represented by c_k , in a data set containing n samples. One can determine the optimal clustering configuration by computing this measurement for each cluster and all samples within the data set. The aim is to obtain a low value of inertia and a minimal number of clusters, as the amount of inertia increases as the number of clusters increases. Therefore, the optimal value for k is determined based on the trade-off between the minimum number

of clusters and the level of inertia. The elbow point, which is the point of diminishing returns, is considered the optimal choice for k , as further increases in the number of clusters do not result in significant reductions in the value of inertia.

In this study, we utilized the average to determine each cluster's features and target variable representative, which would be used for feature selection and training DL models. Fig. 2 illustrates that for a given cluster i with n stocks and m features each, the representative of the cluster would be calculated by taking the average of feature j and the target variable for all n stocks using Eqs. (3) and (4). The stocks within each cluster are highly correlated and similar, meaning their features and target variables behave similarly. Therefore, it is reasonable to use the average of each feature and target variable across all stocks in a given cluster as representative of the cluster.

$$\text{Representative feature of cluster } i = \frac{\sum_{j=1}^n F_{ji}}{n} \quad \forall j = 1, 2, 3, \dots, m \quad (3)$$

$$\text{Representative target variables of cluster } i = \frac{\sum_{i=1}^n \text{Return}_i}{n} \quad (4)$$

3.2.2. Feature selection method

This paper's proposed feature selection method uses the neural network learning process directly to determine optimal features

that have the most effect on the representative target variable of a given cluster. The standardized input features of the DL model are representative features of a given cluster. The standardization scale in Eq. (5) is used to scale the features to have the same mean. The output weights determine the importance of each feature on the target variable. A higher weight indicates a more significant impact on the target variable.

$$X_{\text{standard}} = \frac{X - E(X)}{\sigma} \quad (5)$$

The flowchart of our proposed feature selection method is illustrated in Fig. 3. Each output weight of the output layer is considered as a coefficient for each feature that is multiplied by the desired feature. The resulting sum is represented as $\hat{Y}$. The difference between the predicted value of $\hat{Y}$ and the actual value of the target variable is calculated using the mean squared error (MSE) function, which serves as the neural network loss function. During the backpropagation process, the gradient of the loss function is calculated, and the neural network weights are updated. After a certain number of iterations, the feature with the most significant impact on the target variable is expected to have more weight in the output layer.

3.2.3. Hyperparameter optimization method

The selection of appropriate hyperparameters for ML models has always made it one of the formidable challenges due to the

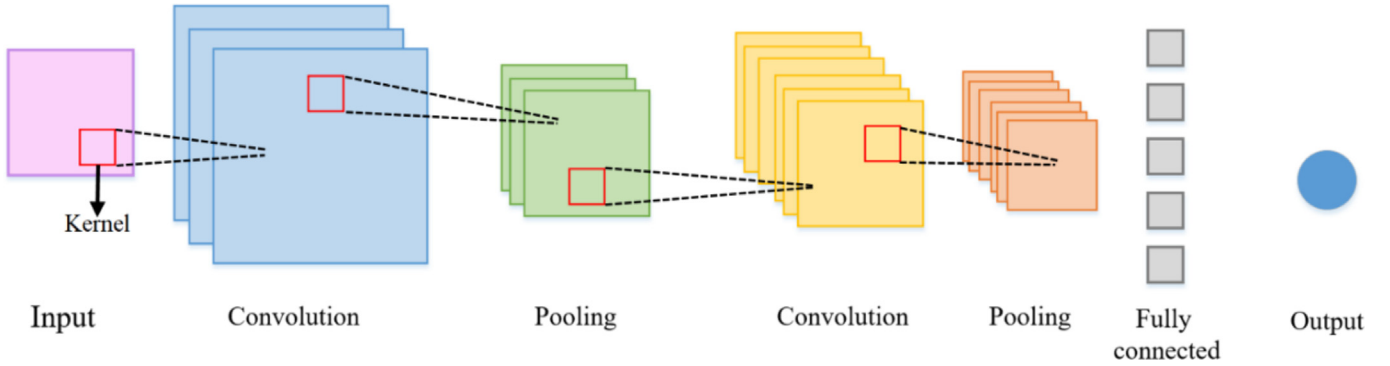


Fig. 4. Typical CNN network structure.

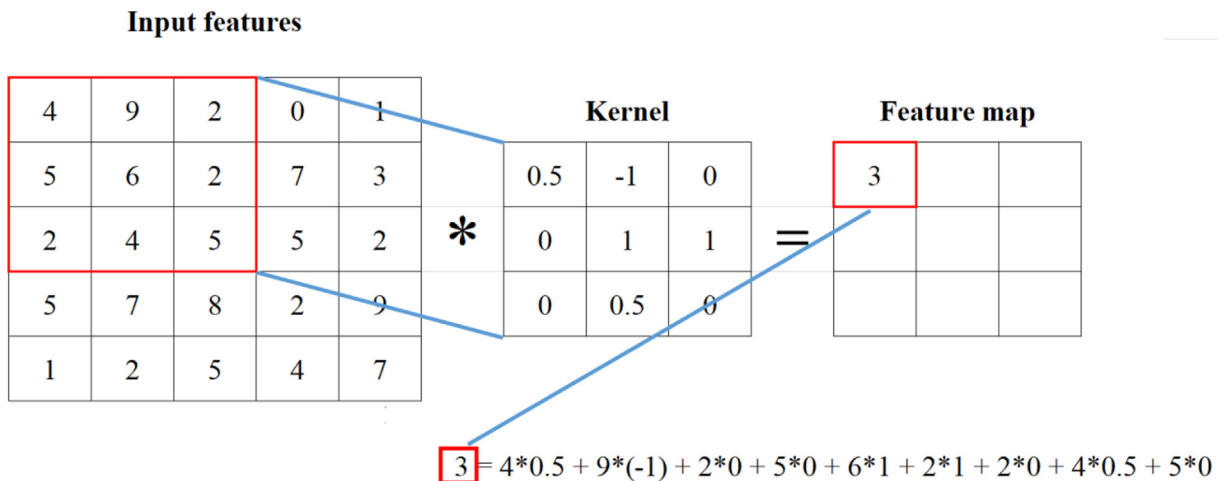


Fig. 5. Operation of a convolutional layer.

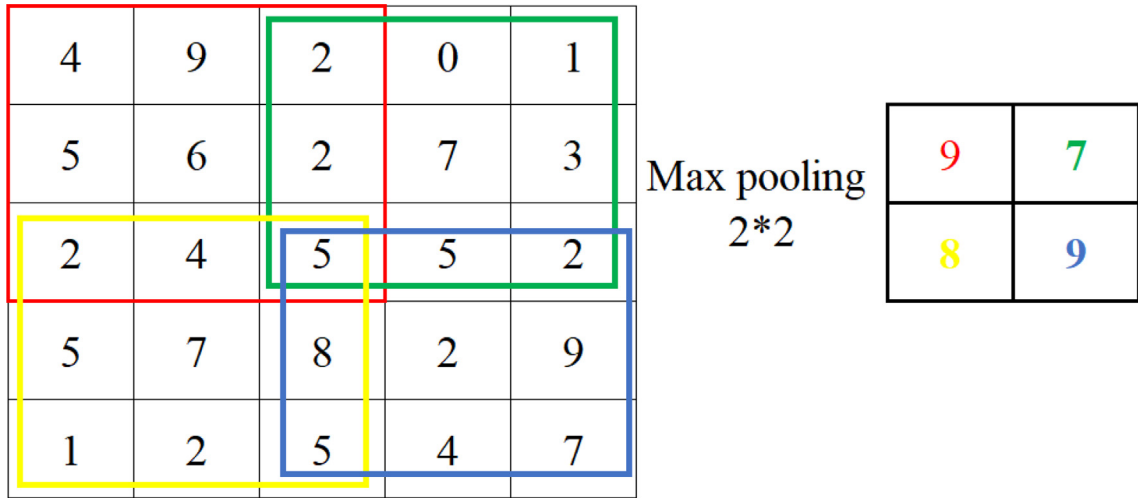


Fig. 6. Operation of a pooling layer.

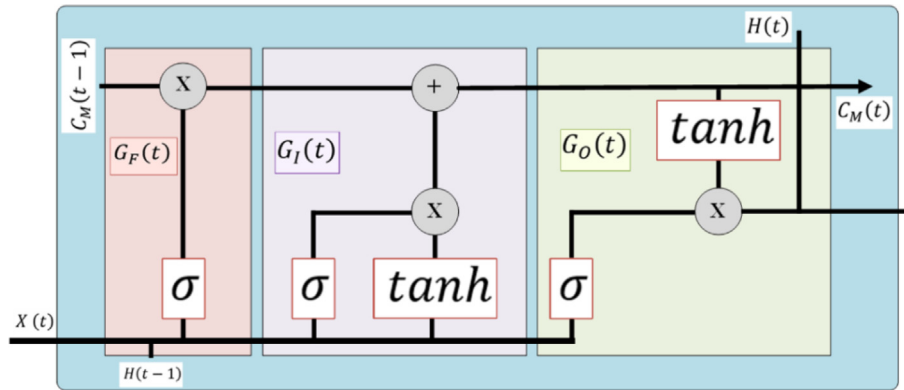


Fig. 7. Typical LSTM network structure.

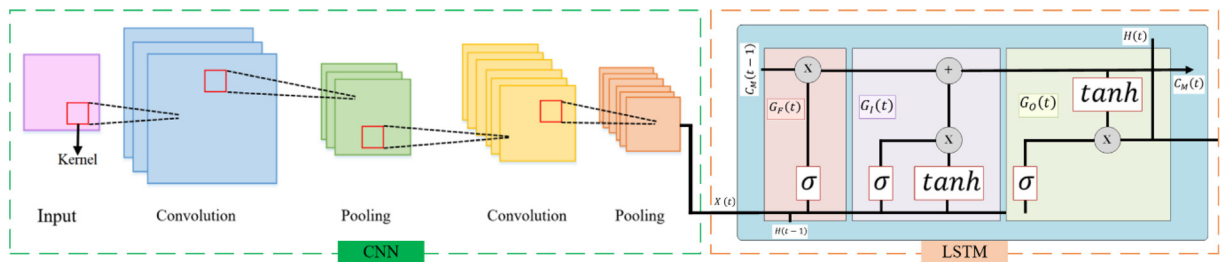


Fig. 8. CNN-LSTM network structure.

large number of hyperparameters that a model may have and the great variety of each hyperparameter. On the other hand, the optimal selection of hyperparameters has a direct effect on the accuracy of the prediction model. In this regard, the PSO is one of the best choices since it enables parallel executions to improve efficiency, particularly for the DL models that often require massive training time (Yang and shami, 2020).

The PSO, originally introduced by Kennedy and Eberhart (1995) takes inspiration from the social conduct of birds. Its popularity stems from its simplicity, minimal parameters, and ease of computation, which renders it a top preference in addressing optimization problems. Compared to other commonly employed meta-

heuristic algorithms, such as genetic algorithms, the PSO lacks certain added operations, such as crossover and mutation (Yang and shami, 2020). The PSO entails each particle communicating with each other to identify and update the present global optimum in each iteration, ultimately leading to the final optimum. Recently, researchers have explored the application of the PSO in the realm of neural networks, which also serves as our incentive in this paper (Lorenzo et al., 2017). Our study utilizes the PSO to automatically discover the optimal model architecture and corresponding hyperparameters, minimizing the necessity for human intervention.

In each iteration of the PSO, the neural network is trained based on the particular hyperparameters chosen by the PSO and the rel-

Table 3

The benchmark models' descriptions to assess the effectiveness of our proposed model.

Part	Benchmarks	Description	Evaluation measures
Feature selection	Without feature selection	All features are used in models	MSE
	Pearson correlation	The top five features that exhibit the strongest correlation with the representative target variable of the cluster have been chosen.	
	Forward stepwise random forest model selection algorithm (FSRF)	The method proposed by Lin (2021)	
Part	Benchmarks	Description	Evaluation measures
Return prediction	Conventional training	Trained for each stock separately and then predicting the return of that stock	MAE, MSE, RMSE, HR%, HR+%, and HR-%
	TCluster-based training	Trained on representative of three clusters and then predicting the return of all stocks.	
Portfolio Optimization	PSO-CNN+1/N	Pre-selection of stocks by the PSO-CNN and selected stocks with equal weight in the portfolio	In terms of return: Arithmetic expected return, Geometric expected return, min, and max In terms of risk: Kurtosis, Standard Deviation, Semi-Deviation, VaR_99%, CVaR_99%, VaR_95%, CVaR_5%, Sortino ratio, Coefficient of variation, Maximum drawdown, and Sharpe ratio
	CNN-LSTM+1/N	Pre-selection of stocks by the CNN-LSTM and selected stocks with equal weight in the portfolio	
	LSTM+1/N	Pre-selection of stocks by the LSTM and selected stocks with equal weight in the portfolio	
	CNN+1/N	Pre-selection of stocks by the CNN and selected stocks with equal weight in the portfolio	
	CNN-LSTM+MVF	Pre-selection of stocks by the CNN-LSTM and the MV model with predicted return by the CNN to determine the optimal stock weight	
	LSTM+MVF	Pre-selection of stocks by the LSTM and the MV model with predicted return by the LSTM to determine the optimal stock weight	
	CNN+MVF	Pre-selection of stocks by CNN and the MV model with predicted return by the CNN-LSTM to determine the optimal stock weight	
	RandomPrs+MVF (1)	Pre-selection of stocks randomly and the MV model with predicted return by the PSO-CNN to determine the optimal stock weight	
	RandomPrs+MVF (2)	Pre-selection of stocks randomly and the MV model with predicted return by the CNN-LSTM to determine the optimal stock weight	
	RandomPrs+MVF (3)	Pre-selection of stocks randomly and the MV model with predicted return by the LSTM to determine the optimal stock weight	
	RandomPrs+MVF (4)	Pre-selection of stocks randomly and the MV model with predicted return by the CNN to determine the optimal stock weight	

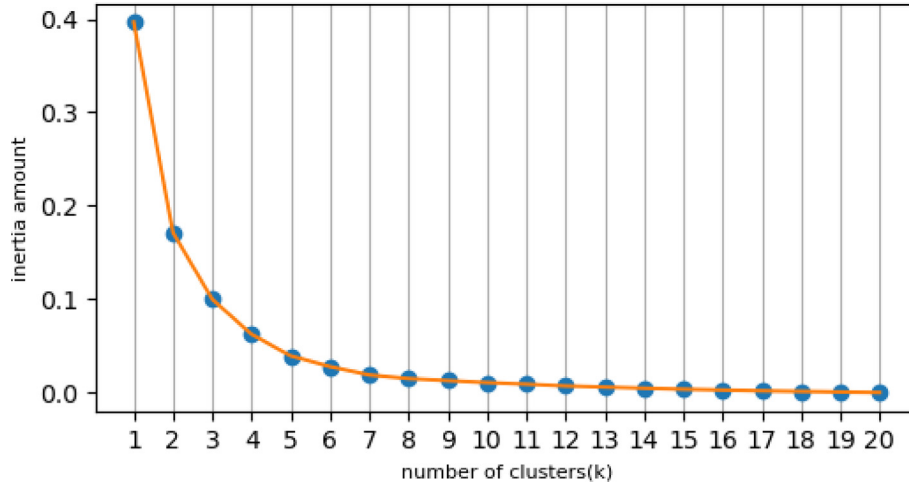


Fig. 9. The inertia index against the number of cluster.

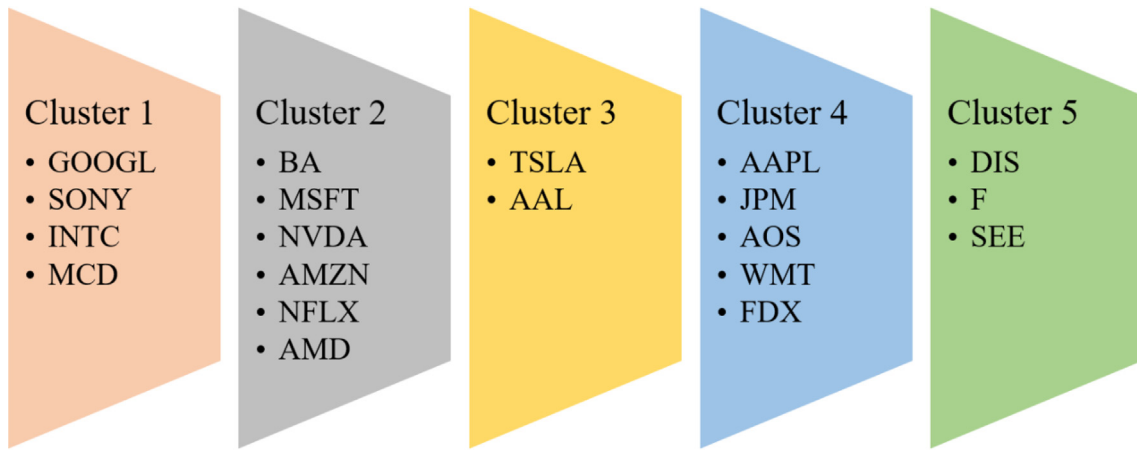


Fig. 10. Clustering result.

evant data in the current cluster. Then the trained network is utilized to predict the return of the stocks within the cluster. The fitness function of the PSO (Eq. (7)) measures the average MSE (Eq. (6)) of the predicted stock returns. This process guarantees that the neural network configuration is sufficiently flexible to accurately anticipate future returns of the stocks in the cluster with a minimum forecasting error.

$$MSE_i = \frac{1}{T} \sum_{j=1}^T (r_j - \hat{r}_j)^2 \quad \forall i = 1, 2, 3, \dots, N \quad (6)$$

$$PSO \text{ fitness function} = \frac{\sum_{i=1}^N MSE_i}{N} \quad (7)$$

where T is the number of predicted values, r_j is the real value of return and $\hat{r}_j$ is the predicted return of stock j , and N is the number of stocks in each cluster. The following is a brief presentation of the steps involved in the PSO (Kumar, 2021).

Step 1. Initialize the swarm with M particles; each particle has a position (X) and a velocity (V) vector.

Step 2. Evaluate the fitness function of each particle at its current position.

Step 3. Identify the best solution (global best) from the fitness values of all particles in the swarm.

Step 4. Update each particle's personal best (PB) based on its current position and fitness value.

Step 5. Update the global best (GB) by selecting the particle with the best fitness value from the entire swarm.

Step 6. Update the velocity of each particle using the following equation:

$V_{new} = w * V_{old} + c_1 * rand() * (PB - X) + c_2 * rand() * (GB - X)$, where w is the inertia weight, c_1 and c_2 are the acceleration coefficients, and $rand()$ generates a random number.

Step 7. Update the position of each particle using the new velocity.

Step 8. Evaluate the fitness function at the new position of each particle.

Step 9. If the stopping criterion is met, stop. Otherwise, go to step 3.

3.2.4. Predictive models

This sub-section elaborates on three DL models, including the CNN, the LSTM, and the CNN-LSTM, and their relevance to predicting stock returns. The discussion covers each network's specific features and operations in the return prediction.

3.2.5. CNN

CNN was first introduced by Lecun et al. (1998) as a type of feedforward deep neural network that utilizes convolution opera-

tions to significantly reduce data dimensionality and represent the original data more abstractly (Gandhmal and Kumar, 2019). A typical CNN structure consists of two major components: convolution and pooling layers. The fundamental idea behind the CNN is to generate multiple filters to extract data features and topological structure features hidden within the data by layer-by-layer convolution and pooling operations performed on input data. The structure of a typical CNN network is illustrated in Fig. 4.

The convolution layer abstractly represents the original data by convolving the information in the receptive field via a convolution kernel of suitable size. The feature map of the convolutional layer for input data x can be expressed as follows (Eq. (8)):

$$c = f(x \otimes w + b) \quad (8)$$

where c denotes the convolution output value, $\otimes$ stands for convolution operation, w represents the convolution kernel weight vector, b indicates the bias, and $f(\cdot)$ refers to the activation function, which could either be ReLU or tanh. The operation of a convolutional layer, which involves using a kernel to convert the input into a feature map, is illustrated in Fig. 5.

As the feature dimensions extracted by the convolution layer are typically very high, a pooling layer is incorporated after the convolution layer to tackle this problem and reduce model training costs to retain robust features while discarding others. The operations carried out in the pooling layer are depicted in Fig. 6.

3.2.6. LSTM

The LSTM is a specific type of RNN proposed by Hochreiter and Schmidhuber (1997). This network architecture effectively overcomes the issue of vanishing gradients which is a well-known problem in traditional RNNs. The LSTM architecture comprises recurrent network units that maintain values for short and long times, use memory cells to store data, and are better at identifying and exploiting long-term dependency. The memory cell plays a crucial role in the LSTM training and performance by storing the temporal information of the previous prediction and propagating it through the network if necessary. Each layer of the LSTM comprises three control gates: the input gate, forget gate, and the output gate. The information stored in memory is dependent on the high sigmoid activation results. The sigmoid layers facilitate outputting numbers within the interval $[0, 1]$, representing a proportion of input information to be allowed through. If the input gate has high activation (closer to one), the information will be stored in the memory cell. Similarly, if the output gate exhibits high activity, it will forward the information to the following neuron. The forget gate will clear the memory cell if it has high activation. The structure of a typical LSTM is depicted in Fig. 7. Overall, the use of LSTM networks provides several benefits, including improved accuracy in predicting future stock returns, the ability to handle variable-length sequences, and the capacity to capture long-term dependencies in the data.

Suppose we have information x_t at time t . The LSTM layer decides on the old information that should be forgotten by outputting a number within the range of $[0,1]$ known as f_t in Eq. (9).

$$f_t = \sigma_1(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (9)$$

where h_{t-1} is the output in time $t - 1$, W_f and b_f is the weight matrices and the bias of the forget gate.

Then, x_t processing before it is stored in the cell state according to Eq. (10). Its value is calculated by the input gate, which also generates a vector of candidate values labeled as $\tilde{C}_t$ through a tanh layer simultaneously state according to Eq. (11). These values are used to update the existing cell state C_t according to Eq. (12).

Table 4
The results of implementing feature selection methods for each cluster.

Features	Cluster 1				Cluster 2				Cluster 3				Cluster 4				Cluster 5			
	Correlation	FSRF	Our proposed method	All	Correlation	FSRF	Our proposed method	All	Correlation	FSRF	Our proposed method	All	Correlation	FSRF	Our proposed method	All	Correlation	FSRF	Our proposed method	All
MACD	+			+	+		+	+	+		+	+	+		+	+	+		+	+
PPO				+				+				+								
ROC				+				+				+								
StochRSI				+				+				+								
RSI				+				+				+								
ATR				+				+				+								
PSAR				+				+				+								
Stochastic				+				+				+								
EMA				+				+				+								
SMA				+				+				+								
LSTM				+				+				+								
(MSE)	0.003	0.003	0.0012	0.006	0.005	0.004	0.0011	0.007	0.0039	0.0036	0.0029	0.010	0.0032	0.0028	0.0014	0.007	0.006	0.004	0.0011	0.009

Table 5

The list of parameters used for implementation of the PSO.

Parameter	Value
Convolution1 number of filters	(32, 256) with length 32
Convolution2 number of filters	(32, 256) with length 32
Learning rate	(0.00001, 0.0001) with length 0.00001
Social coefficient (c1)	0.7
Cognitive coefficient (c2)	0.7
Inertia weight (ω)	0.4
Maximum iteration	40

$$i_t = \sigma_2(W_i[h_{t-1}, x_t] + b_i) \quad (10)$$

$$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c) \quad (11)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad (12)$$

where (W_i, b_i) and (W_c, b_c) are the weight matrices and the input gate and memory cell state biases, respectively. Finally, the output gate, which Eqs define. (13 and 14).

$$o_t = \sigma_3(W_o[h_{t-1}, x_t] + b_o) \quad (13)$$

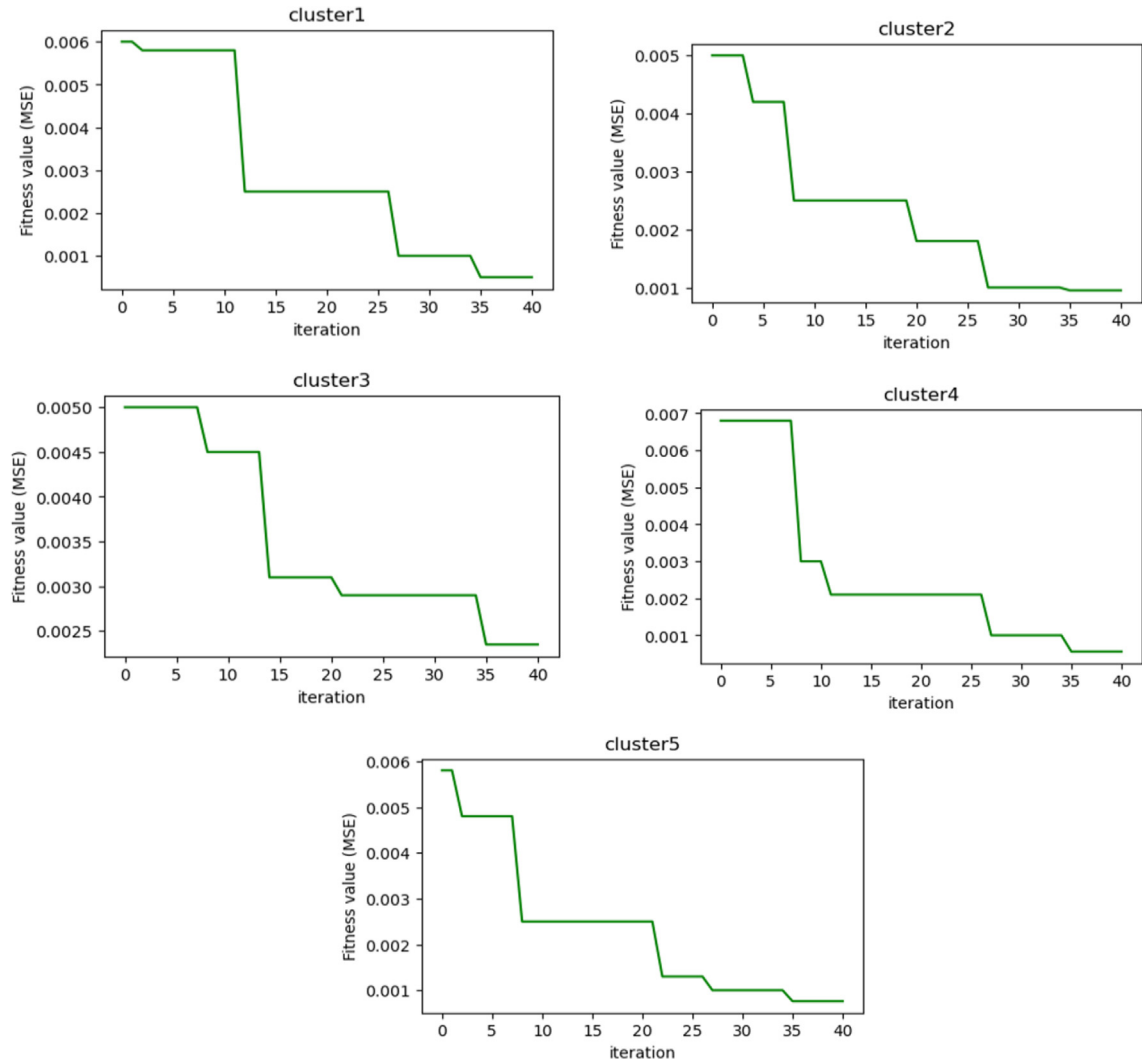
$$h_t = o_t * \tanh(C_t) \quad (14)$$

The weight matrix and bias of the output gate denoted as (W_o, b_o) , play a role in deciding which portion of the cell state is to be outputted.

3.2.7. CNN-LSTM

The CNN-LSTM combines the feature extraction capability of the CNN networks with the temporal learning ability of the LSTM networks. This integrated network can effectively predict complex data by leveraging the strengths of both architectures. Previous studies, such as [Kim and Cho \(2019\)](#), and [Livieris et al. \(2020\)](#), have successfully utilized the CNN-LSTM for challenging prediction problems. The CNN-LSTM method involves two main steps:

The first step is to use a CNN to extract features from the input data. The CNN helps to identify the important features in the input data and reduce the dimensionality of the data. The second step is to use an LSTM network to process the feature maps generated by the CNN. In the case of the CNN-LSTM method, the feature maps generated by the CNN are treated as a sequence of inputs to the LSTM. The LSTM network is trained to learn the temporal dependencies between these inputs and to generate an output for each input in the sequence. The final output of the LSTM is typically a prediction or classification for the entire input sequence. The structure of the CNN-LSTM network is illustrated in [Fig. 8](#).

**Fig. 11.** Variations of fitness function during iterations until convergence.

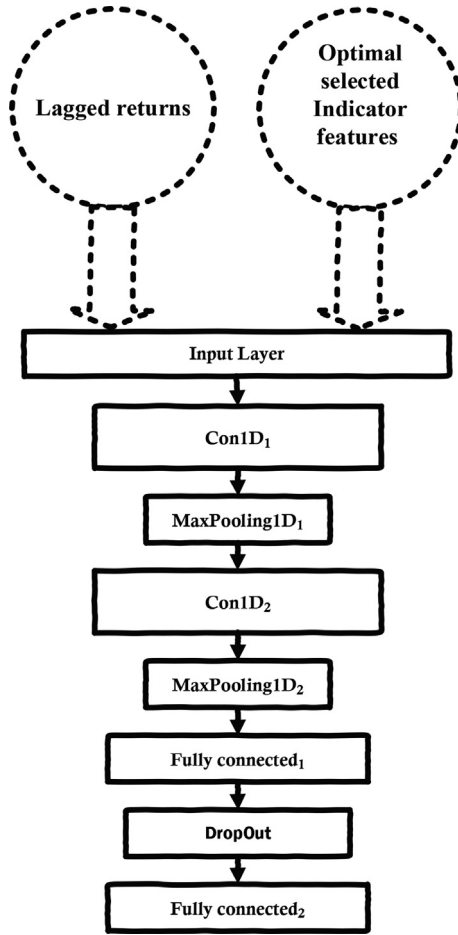


Fig. 12. The PSO-CNN structure in our proposed model.

3.3. Portfolio optimization methodology

Our proposed model outlines a two-step approach to portfolio optimization. The first step involves utilizing the ranking-based pre-selection approach to pre-select seven stocks with anticipated

high predicted returns for the upcoming trading day. The second step applies the MVF model to determine the optimal allocation ratio of each selected stock into the portfolio for the next trading day.

3.3.1. Asset pre-selection

The pre-selection of stocks is done based on their predicted return for the next period, such as Wang et al. (2020), and Ta et al. (2020). First, the stocks are sorted based on their predicted return, then the stocks with the highest predicted return are selected according to the portfolio size.

3.3.2. MVF

The MV model is the backbone of modern portfolio theory (MPT) and offers a solution to the optimal portfolio selection issue. This model quantifies investment return and risk using expected return and variance. A subsequent model that builds upon this foundation is the MVF approach developed by Yu et al. (2020) and Ma et al. (2021). The MVF model enables the optimal allocation of selected stocks and portfolio rebalancing based on predicted return and prediction errors for the following trading day. To elaborate, the model functions as follows (Eqs. (15) and (16)):

$$\min \sum_{i,j=1}^N w_i w_j \delta_{ij} - \sum_{i=1}^N w_i \bar{r}_i - \sum_{i=1}^N w_i \bar{e}_i \quad (15)$$

$$S.T. \begin{cases} \sum_{i=1}^N w_i = 1 \\ 0 \leq w_i \leq 1 \end{cases} \quad (16)$$

where N is the number of stocks in the portfolio, w_i is the proportion of stock i in portfolio, $\bar{r}_i$ is the predicted return of stock i at time t . δ_{ij} is the covariance between stock i and j , which in this study is calculated by the historical data of the stocks for four years before the test period and is updated every six months during the test period. $\bar{e}_i$ is the average prediction error (e_i) of the stock i , which is calculated for the last twenty days before the day t . e_i is obtained by $e_i = r_i - \bar{r}_i$, where r_i is the actual return of stock i .

Table 6
Applied hyperparameters with the PSO-CNN model.

	Categories	Cluster 1	Cluster 2	Cluster 3	Cluster 4	Cluster 5
Con1D ₁	Filters (determined by the PSO)	32	32	32	32	32
	kernel_size	1	1	1	1	1
	activation	selu	selu	selu	selu	selu
MaxPooling1D ₁	pool_size	1	1	1	1	1
Con1D ₂	Filters (determined by the PSO)	32	32	64	32	32
	kernel_size	1	1	1	1	1
	activation	selu	selu	selu	selu	selu
MaxPooling1D ₂	pool_size	1	1	1	1	1
Fully connected ₁	Neurons	128	128	128	128	128
	activation	linear	linear	linear	linear	linear
DropOut	DropOut_rate	0.25	0.2	0.5	0.5	0.5
Fully connected ₂	Neurons	1	1	1	1	1
	activation	tanh	tanh	tanh	tanh	tanh
Fitting parameters	Optimizers	Adam	Adam	Adamax	Adamax	Adamax
	learning_rate (determined by the PSO)	7e-05	9e-05	9e-05	5e-05	5e-05
	epochs	60	150	120	120	120
	batch_size	128	256	256	256	256
PSO-Fitness		0.0005	0.0008	0.0021	0.0006	0.0006

Table 7

Comparison of the prediction performance of the PSO-CNN in two training method. Our proposed Training method: Training on the representative of each cluster and then predicting the returns of stocks within the same cluster. Conventional training: Trained for each stock separately and then predicted the return of that stock.

stocks	Our proposed Training method			Conventional training			Diebold-Mariano Test Result	
	MSE	RMSE	MAE	MSE	RMSE	MAE	DM-value	P-value
TSLA	0.0021	0.0458	0.0259	0.0024	0.0490	0.0342	2.6561	0.0081
GOOGL	0.0004	0.0200	0.0110	0.0005	0.0224	0.0154	0.8675	0.3861
AAPL	0.0006	0.0245	0.0159	0.0008	0.0283	0.0191	1.1399	0.2548
BA	0.0011	0.0332	0.0234	0.0018	0.0424	0.0265	-1.3695	0.1714
MSFT	0.0005	0.0224	0.0150	0.0005	0.0224	0.0151	0.0509	0.9594
DIS	0.0006	0.0245	0.0159	0.0006	0.0245	0.0159	0.8506	0.3954
SONY	0.0004	0.0200	0.0117	0.0006	0.0245	0.0160	1.5025	0.1336
JPM	0.0007	0.0265	0.0168	0.0007	0.0265	0.0172	1.0165	0.3099
AOS	0.0005	0.0224	0.0159	0.0005	0.0224	0.0158	-0.8882	0.3748
WMT	0.0004	0.0200	0.0108	0.0004	0.0200	0.0109	-0.0285	0.9773
NVDA	0.0010	0.0316	0.0230	0.0012	0.0346	0.0249	0.6719	0.5020
AMZN	0.0005	0.0224	0.0157	0.0005	0.0224	0.0158	0.0864	0.9312
FDX	0.0007	0.0265	0.0170	0.0007	0.0265	0.0183	1.5013	0.1339
NFLX	0.0006	0.0245	0.0171	0.0008	0.0283	0.0209	3.3500	0.0009
AMD	0.0007	0.0265	0.0168	0.0014	0.0374	0.0276	5.3279	0.0000
F	0.0010	0.0316	0.0218	0.0011	0.0332	0.0221	1.9877	0.0474
AAL	0.0022	0.0469	0.0276	0.0026	0.0510	0.0325	1.0219	0.3073
INTC	0.0006	0.0245	0.0168	0.0008	0.0283	0.0180	0.4299	0.6674
MCD	0.0004	0.0200	0.0116	0.0004	0.0200	0.0116	0.2330	0.8158
SEE	0.0007	0.0265	0.0166	0.0007	0.0265	0.0168	0.9864	0.3244
Avg.	0.0009	0.0270	0.0173	0.0010	0.0295	0.0197	1.0697	0.3851

Table 8

Prediction performance of different DL models.

Model	Measure	Cluster1	Cluster2	Cluster3	Cluster4	Cluster5
PSO-CNN	MAE	0.0131	0.0150	0.0260	0.0144	0.0146
	MSE	0.0004	0.0006	0.0021	0.0006	0.0006
	RMSE	0.0200	0.0245	0.0458	0.0237	0.0245
	HR%	51.2366	50.2151	49.8095	51.0605	50.2509
	HR+%	56.3323	48.7924	49.6236	55.2312	52.1013
LSTM	HR-%	45.1196	51.4371	49.7318	46.5903	49.2671
	MAE	0.0258	0.0207	0.0358	0.0235	0.0209
	MSE	0.0013	0.0010	0.0029	0.0013	0.0011
	RMSE	0.0361	0.0308	0.0539	0.0361	0.0332
	HR%	49.5699	52.0430	47.9570	50.1505	50.7527
CNN-LSTM	HR+%	45.5724	74.1680	43.9912	54.5512	36.9520
	HR-%	54.2338	28.2776	52.2855	45.4903	65.1783
	MAE	0.0142	0.0204	0.0347	0.0151	0.0193
	MSE	0.0006	0.0011	0.0027	0.0006	0.0009
	RMSE	0.0245	0.0332	0.0520	0.0245	0.0300
CNN	HR%	51.0215	49.3548	48.9247	50.3226	50.3226
	HR+%	62.9262	46.2502	54.5659	79.8602	81.3053
	HR-%	36.7469	52.4406	45.1981	18.0057	19.1835
	MAE	0.0137	0.0208	0.0335	0.0139	0.0187
	MSE	0.0005	0.0010	0.0025	0.0006	0.0008
	RMSE	0.0225	0.0302	0.0496	0.0231	0.0283
	HR%	52.8495	50.3226	50.1076	49.5054	48.6021
	HR+%	84.4092	39.4182	45.3336	55.1497	52.8529
	HR-%	50.34	46.23	49.32	48.6	3.49

3.4. Benchmark models and evaluation measures

Benchmark models have been assessed in every part where our proposed model has a contribution. The specifics of the benchmark models employed and the measures used to evaluate them are presented in Table 3. The performance evaluation measures specifics can be found in Appendix B.

4. Numerical experiments & results

This section illustrates the implementation results of various models across four sub-sections. Firstly, we evaluate the outcomes of the stock clustering method. Next, we evaluate the effectiveness of the proposed feature selection method. Following this, we discussed the results of return prediction and portfolio optimization.

Table 9
Performance of different models without transaction cost.

Model	Our proposed model	PSO-CNN + 1/N	CNN_LSTM + 1/N	LSTM + 1/N	CNN + 1/N	CNN_LSTM + MVF	LSTM + MVF	CNN + MVF	RandomPrS + MVF (1)	RandomPrS + MVF (2)	RandomPrS + MVF (3)	RandomPrS + MVF (4)
Panel A: Daily return measures												
Expected Return	0.0071	0.0032	0.0024	0.003	0.0021	0.003	0.0057	0.0046	0.002	0.0008	0.0017	0.001
Panel B: Daily risk measures												
Standard Deviation	0.0389	0.0218	0.0212	0.0234	0.0207	0.0411	0.0356	0.036	0.0327	0.031	0.0317	0.032
VaR_1	0.0834	0.0476	0.0468	0.0514	0.0461	0.0926	0.0771	0.0791	0.074	0.0714	0.0722	0.0735
CVaR_1	0.1323	0.0719	0.082	0.0878	0.0698	0.1602	0.1168	0.1207	0.1375	0.1398	0.1265	0.1201
Coefficient of variation	5.4589	6.7465	8.9052	7.7971	10.0914	13.6444	6.2246	7.8177	16.1292	37.5183	18.7728	32.5977
Maximum drawdown	1.5008	1.9611	2.5987	1.7547	2.0408	1.5125	2.3505	2.0382	2.5979	2.6614	2.7178	2.6614
Panel C: Annualized risk-return measures												
Annual return	4.9007	1.244	0.8097	1.1155	0.669	1.1205	3.158	2.1517	0.6589	0.2297	0.5256	0.2781
Annual SD	0.615	0.3454	0.3345	0.3701	0.3273	0.6496	0.5626	0.5689	0.5169	0.4909	0.5019	0.5062
Annual sortino	9.0543	2.2984	1.496	2.061	1.236	2.0703	5.8347	3.9754	1.2174	0.4244	0.9711	0.5138

4.1. Clustering results

The K-means method employed in this study utilized the mean and variance of the input features and asset returns during the training period as clustering criteria. The relationship between the number of clusters and the inertia index is depicted in Fig. 9. It is evident that the elbow point falls on the number of five clusters, indicating a balance between the cluster number and the inertia value. As such, clustering is conducted using five clusters. Fig. 10 depicts the stocks that belong to each cluster.

4.2. Feature selection results

Appendix C provides details of the LSTM used to evaluate the validity of feature selection methods. The MSE on validation data is used as the comparison measure. The results show that when all features are included as inputs to the model, the prediction error rate is higher than when only the selected features from the feature selection methods are used as inputs.

Thus, it can be concluded that each feature selection method contributes to reducing the neural network prediction error. For instance, using all the features in Table 4 as the LSTM input for cluster 1 results in an MSE of 0.0066, while using only the MACD, the ROC, the ATR, the PSAR, and the EMA, selected by the proposed method, as the LSTM input leads to a reduced MSE of 0.0045. Based on the LSTM model's MSE results, it can be inferred that the prediction error is lower when trained with features selected by the proposed method compared to other method.

4.3. Return prediction results

The PSO-CNN structure utilized in our proposed model is exhibited in Fig. 12. The hyperparameters for the number of filters in Conv1D1 and Conv1D2 layers and the learning rate amount were optimized utilizing the PSO. A summary of the parameters utilized in the PSO implementation is illustrated in Table 5, wherein the stop criteria are set at a maximum iteration of 40. Fig. 11 displays the variations in fitness function for each cluster during 40 iterations individually. It can be concluded that according to the number and size of changes in the fitness function, the use of the PSO is effective and justified. Table 6 illustrates the network hyperparameters for each cluster, such as the learning rate, which was optimized by PSO and set as 0.0005, 0.0008, 0.0021, 0.0006, and 0.0006 for clusters one, to five, respectively.

The Diebold-Mariano test results in Table 7 illustrate that in most cases, the p-value is greater than 5%, indicating no significant difference between the two prediction models. However, for some stocks such as TSLA, NFLX, and AMD, there is a significant difference between the two training methods, and our proposed model outperforms. This is because using a common prediction model for the assets of a cluster increases generalizability and generality compared to when a separate model is trained for each asset, which can result in better resistance against overfitting. Therefore, it can be concluded that our proposed model not only reduces the number of prediction model training and computational complexity but also performs better against overfitting.

To assess the effectiveness of the PSO-CNN model, we compared it with other DL models such as the LSTM, the CNN-LSTM, and the CNN, whose structures are listed in Appendix C. Our proposed training and feature selection methods were applied to train these models. Table 8 presents the returns prediction results, which demonstrate that the PSO-CNN model exhibits the lowest error compared to other models. Specifically, the PSO-CNN model showed reduced MAE, MSE, and RMSE compared to the CNN model. Additionally, accuracy indexes such as the HR%, HR+%,

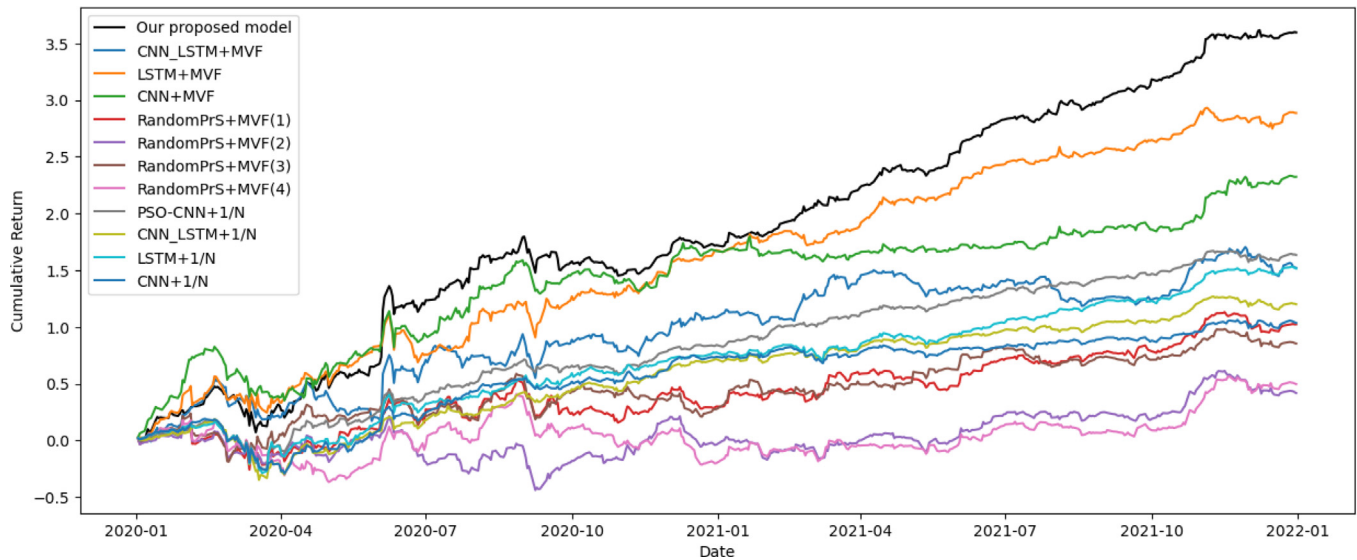


Fig. 13. Cumulative portfolio return without transaction cost.

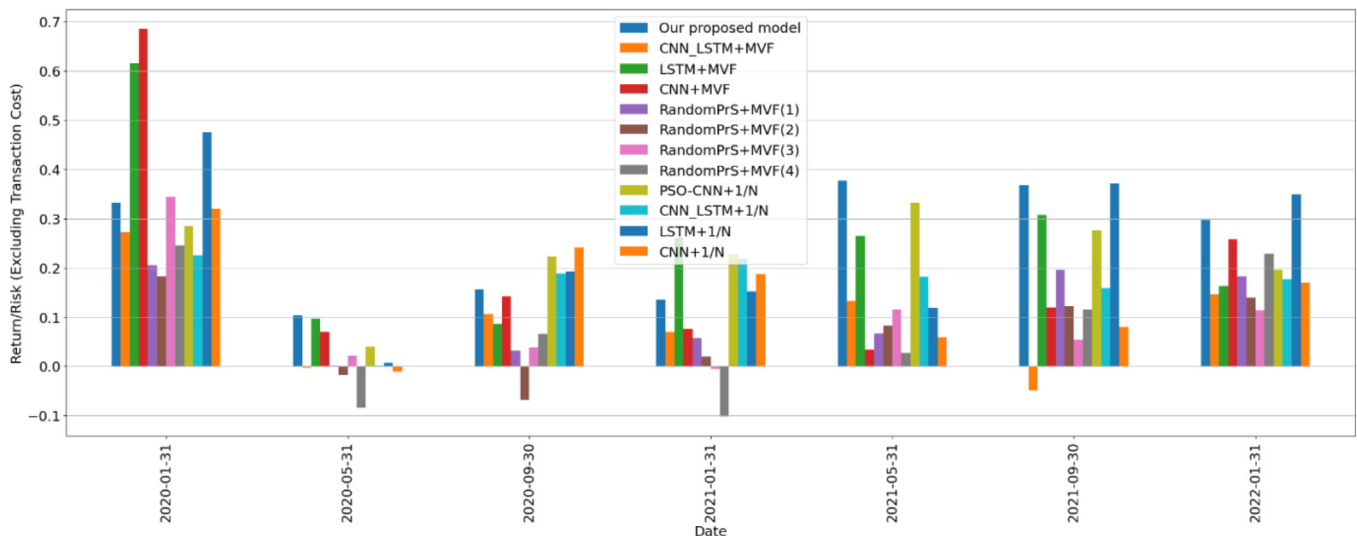


Fig. 14. Return-risk ratio of every two month without transaction costs.

and HR-% indicate that the PSO is highly effective for hyperparameter tuning.

4.4. Portfolio optimization results

In this sub-section, the performance of different optimal portfolios is measured and compared using nine criteria: The expected return, standard deviation, value at risk (VaR), conditional value at risk (CVaR), Sortino ratio, coefficient of variation, maximum drawdown, cumulative return, and risk-return ratio. Buying and selling behaviors of a typical investor are simulated by assuming an investor decides to buy or sell a certain proportion of each asset before each trading day. This is done to achieve the optimal asset ratio in the portfolio. As transaction costs play an essential role in quantitative investment research (Box et al., 2015), we compare the performance of each model based on three scenarios: initially without considering transaction costs and then with 1% and 2% transaction costs. Table 9 and Tables E1 and E2 in Appendix E compare the model's performance without and with transaction costs of 1% and 2%. Specifically, Panels A, B, and C describe daily return

measures, daily risk measures, and annualized risk-return measures, respectively.

(1) Panel A: Comparison of daily return measures

The highest daily mean return of 0.0071 is displayed in Panel A of Table 9, achieved by our proposed model. The LSTM + MVF model follows behind with a daily mean return of 0.0057, while the CNN + MVF models attain 0.0046. Upon considering transaction costs, Tables E1 and E.2 in Appendix E demonstrate that our proposed model maintains the highest level of performance.

(2) Panel B: Comparison of daily risk measures

In Panel B of Table 9 as well as Tables E1 and E2 in Appendix E, we utilize standard deviation, VaR, CVaR, Sortino ratio, coefficient of variation, and maximum drawdown to measure the daily risks of different models. The outcomes reveal that: (1) The CNN + 1/N model exhibits the lowest standard deviation, VaR, and CVaR, whether or not transaction costs are considered. Our proposed

model exhibits the lowest positive coefficient of variation and maximum drawdown, both without and with transaction costs of 1% and 2%.

(3) Panel C: Comparison of annualized risk-return measures.

Panel C of [Table 9](#) shows that our proposed model has the highest annualized return of 4.9007 and a Sortino ratio of 9.0543. In second place, the LSTM + MVF has the highest annualized return of 3.1580. After considering transaction costs of 1% and 2%, as shown in [Tables E.1 and E.2](#) in [Appendix E](#), our proposed model has the highest annualized return and Sortino ratio.

[Fig. 13](#) displays a visualization of the cumulative returns for each model, excluding transaction costs. Our suggested model exhibits the highest cumulative return, followed by the LSTM + MVF and the CNN + MVF. In [Appendix F](#), [Figures F.1 and F.2](#) depict the cumulative returns of each model with 1% and 2% transaction costs. After considering 1% and 2% transaction costs, this models also display the highest cumulative returns, ranked in the same order.

To ensure that a model's success is not attributed solely to a particular period, a histogram of the risk-return ratio for various models is generated every two months, as observed in earlier research ([Wang et al., 2020](#)). The findings depicted in [Fig. 14](#) demonstrate that almost all models exhibit positive risk-return ratios in the absence of transaction costs. Meanwhile, [Figures F.3 and F.4](#) of [Appendix F](#) demonstrate that some models exhibit negative risk-return ratios when transaction costs of 1% and 2% are accounted for. Nevertheless, our proposed model does not have a negative risk-return ratio in each two-month time period during the test period for different transaction costs. As a result, our proposed model's overall performance is exceptionally strong and not merely coincidental, underscoring its dependability.

5. Conclusion

This paper presents a novel framework for simplifying portfolio construction computations based on return prediction by reducing the number of learning models. Our proposed framework utilizes ML models to predict the average return of stocks within clusters, allowing us to use a general model instead of training separate models for each stock. This reduces the number of learning models and enhances the accuracy and efficiency of the prediction process.

We apply the k-means method to divide stocks into clusters and use feature selection models to select optimal technical indicators for predicting returns. Moreover, we propose a CNN-LSTM-based feature selection method that weighs the importance of features during the learning process, resulting in better performance compared to other deep learning models.

The predicted returns of the stocks are then used to construct the portfolio using the MVF model, which provides the optimal weight of stocks in the portfolio for the next trading period. Our proposed framework offers a promising approach to simplify the

portfolio construction process, and we believe it has significant practical implications for investors and financial institutions.

In summary, our research has yielded the following key findings:

- Training machine learning models using clustered stock information leads to comparable accuracy compared to training on individual stocks in the test set, while being less complex.
- Our evaluation of different deep learning models for yield prediction demonstrates the superiority of the PSO-CNN model based on performance evaluation criteria.
- Our proposed PSO-CNN + MVF model for portfolio construction outperforms other models in terms of both risk and return. Our findings suggest that our approach provides a promising solution for simplifying the portfolio construction process while achieving high accuracy and performance.

While our study focuses on predicting returns using historical data and technical indicators, future studies could incorporate external factors such as the COVID-19 crisis, sentiment analysis, and macroeconomic variables to improve prediction model performance. Additionally, our use of the k-means method for clustering could be compared with other clustering methods to further enhance the proposed framework.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Details of the input features used in the models

See [Table A1](#).

Appendix B. Details of the performance evaluation measures

Accurate forecasting is crucial for maximizing portfolio profitability. To determine the most effective forecasting model, several criteria are commonly used, including mean squared error (MSE), mean absolute error (MAE), and root mean squared error (RMSE). Additionally, other criteria, such as H_R , H_{R+} , and H_{R-} are used to assess the model's ability to predict positive and negative returns, as seen in previous studies [25], [24], [40], [50], [23], and [59]. These metrics are defined as Eqs. (B.1)–(B.6):

$$MAE = \frac{1}{n} \sum_{i=1}^n |r_i - \hat{r}_i| \quad (B.1)$$

$$MSE = \frac{1}{n} \sum_{i=1}^n (r_i - \hat{r}_i)^2 \quad (B.2)$$

Table A1

Describes selected features in more detailed.

Features	Formula	Assumptions
Exponential Moving Average	$EMA_{Today} = \left(Value_{Today} * \left(\frac{Smoothing}{1+Days} \right) \right) + EMA_{Yesterday} * \left(1 - \left(\frac{Smoothing}{1+Days} \right) \right)$	Smoothing = 2
Simple Moving Average	$SMA = \frac{A1+A2+\dots+An}{n}$	A_n = the price of an asset at period n n = the number of total periods
Moving average convergence/divergence	$MACD = EMA_a - EMA_b$	EMA_a : 12 period EMA_b : 26 period
Percentage Price Oscillator	$PPO = \frac{12_period\ EMA - 26_period\ EMA}{26_period\ EMA} \times 100$	EMA = exponential moving average
Price Rate of Change	$ROC = \left(\frac{Closing\ Price_p - Closing\ Price_{p-n}}{Closing\ Price_{p-n}} \right) \times 100$	Closing Price _p = Closing price of most recent period Closing Price _{p-n} = Closing price n periods beforemost recent period
Stochastic Oscillator	$\%K = \left(\frac{C-L14}{H14-L14} \right) \times 100$	C = The most recent closing price L14 = The lowest price traded of the 14 previous trading sessions H14 = The highest price traded during the same 14-day period %K = The current value of the stochastic indicator
Relative Strength Index	$RSI_{step\ one} = 100 - \left[\frac{100}{1 + \frac{Average\ gain}{Average\ loss}} \right]$ $RSI_{step\ two} = 100 - \left[\frac{100}{1 + \frac{(Previous\ Average\ Gain \times 13) + Current\ Gain}{(Previous\ Average\ Loss \times 13) + Current\ Loss}} \right]$	
Average True Range	$TR = \text{Max}[(H - L), \text{Abs}(H - C_p), \text{Abs}(L - C_p)]$ $ATR = \left(\frac{1}{n} \right) \sum_{i=1}^n TR_i$	$TR_i = \text{aparticulartruerangen}$ = time period
Parabolic SAR	$RPSAR = \text{Prior PSAR} + [\text{Prior AF}(\text{Prior EP} - \text{Prior PSAR})]$ $FPSAR = \text{Prior PSAR} - [\text{Prior AF}(\text{Prior PSAR} - \text{Prior EP})]$	RPSAR = Rising PSAR AF = Acceleration Factor, it starts at 0.02 and increases by 0.02, up to a maximum of 0.2, each time the extreme point makes a new low (falling SAR) or high(rising SAR) FPSAR = Falling PSAR EP = Extreme Point, the lowest low in the current downtrend(falling SAR)or the highest high in the current uptrend(rising SAR)
Stochastic RSI	$StochRSI = \frac{RSI - \min[RSI]}{\max[RSI] - \min[RSI]}$	RSI = Current RSI reading min[RSI] = Lowest RSI reading over the last 14 periods (or your chosen lookback interval) max[RSI] = Highest RSI reading over the last 14 periods (or your chosen lookback interval)
Lag1	$\ln\left(\frac{closeprice_i}{closeprice_{i-1}}\right)$	
Lag2	$\ln\left(\frac{closeprice_{i-1}}{closeprice_{i-2}}\right)$	
Features	Formula	Assumptions
Lag3	$\ln\left(\frac{closeprice_{i-2}}{closeprice_{i-3}}\right)$	
Lag4	$\ln\left(\frac{closeprice_{i-3}}{closeprice_{i-4}}\right)$	

Table C1

Details of the structure of DL models used in this paper.

LSTM			CNN-LSTM			CNN		
Layers	Categories	values	Layers	Categories	values	Layers	Categories	values
Lstm1	Unit number	32	Cnn1	Unit number	32	Cnn1	Unit number	64
	activation	tanh		kernel_size	3		kernel_size	2
Lstm 2	Unit number	32		activation	selu		activation	selu
	activation	tanh	MaxPooling	Pooling_size	2	MaxPooling1	Pooling_size	2
Fully connected1	Neuron number	32	Lstm1	Unit number	32	Cnn2	Unit number	32
	activation	linear		activation	selu		kernel_size	3
Dropout1	Drop out rate	0.25	Lstm 2	Unit number	64		activation	selu
Fully connected 2	Neuron number	1		activation	selu	MaxPooling 2	Pooling_size	3
	activation	linear	Fully connected1	Neuron number	128	Fully connected1	Neuron number	128
Other	Optimizers	Adam		activation	linear		activation	linear
	learning_rate	0.001	Dropout1	Drop out rate	0.5	Dropout1	Drop out rate	0.25
	epochs	60	Fully connected 2	Neuron number	1	Fully connected 2	Neuron number	1
	batch_size	256		activation	tanh		activation	tanh
			Other	Optimizers	Adam	Other	Optimizers	Adam
				learning_rate	0.0005		learning_rate	0.0007
				epochs	120		epochs	125
				batch_size	128		batch_size	128

Table E1

Performance of different models with transaction cost (=1%).

Model	Our proposed model	PSO-CNN + 1/N	CNN_LSTM + 1/N	LSTM + 1/N	CNN + 1/N	CNN_LSTM + MVF	LSTM + MVF	CNN + MVF	RandomPrS + MVF (1)	RandomPrS + MVF (2)	RandomPrS + MVF (3)	RandomPrS + MVF (4)
Panel A: Daily return measures												
Expected Return	0.0061	0.0013	0.0007	0.0018	0.0006	0.002	0.0051	0.0037	0.0009	−0.0003	0.0007	0
Panel B: Daily risk measures												
Standard Deviation	0.039	0.0219	0.0212	0.0234	0.0207	0.0412	0.0356	0.036	0.0327	0.0311	0.0318	0.032
VaR_1	0.0846	0.0496	0.0485	0.0527	0.0476	0.0938	0.0778	0.08	0.0751	0.0726	0.0732	0.0745
CVaR_1	0.1328	0.0738	0.0836	0.089	0.0712	0.1617	0.1179	0.1211	0.1383	0.1412	0.1273	0.1209
Coefficient of variation	6.4009	16.7025	28.3989	13.0917	34.6426	20.6635	7.0332	9.7235	35.7615	−111.5616	43.8941	−931.4316
Maximum drawdown	1.5008	1.9849	2.6416	1.768	2.0657	1.5162	2.3725	2.05	2.6102	2.6772	2.7319	2.6746
Panel C: Annualized risk-return measures												
Annual return	3.5603	0.387	0.2046	0.563	0.1612	0.6447	2.5353	1.518	0.2565	−0.0673	0.1983	−0.0086
Annual sortino	6.5798	0.7152	0.3781	1.0405	0.2979	1.1914	4.6855	2.8053	0.474	−0.1243	0.3664	−0.0158

Table E2

Performance of different models with transaction cost (=2%).

Model	Our proposed model	PSO-CNN + 1/N	CNN_LSTM + 1/N	LSTM + 1/N	CNN + 1/N	CNN_LSTM + MVF	LSTM + MVF	CNN + MVF	RandomPrS + MVF(1)	RandomPrS + MVF(2)	RandomPrS + MVF(3)	RandomPrS + MVF(4)
Panel A: Daily return measures												
Expected Return	0.0051	0.0007	0.0002	0.0016	0.0002	0.001	0.0045	0.0029	−0.0002	−0.0014	−0.0002	−0.001
Panel B: Daily risk measures												
Standard Deviation	0.039	0.0219	0.0212	0.0234	0.0207	0.0412	0.0356	0.036	0.0326	0.0311	0.0318	0.032
VaR_1	0.0857	0.0502	0.0491	0.0528	0.048	0.0949	0.0785	0.0809	0.0761	0.0737	0.0741	0.0755
CVaR_1	0.1333	0.0766	0.0841	0.0891	0.0717	0.1632	0.119	0.1275	0.1392	0.1311	0.1282	0.1218
Coefficient of variation	7.6932	31.2563	138.2939	14.5127	117.3065	40.2748	7.9863	12.6234	−189.5435	−22.8935	−147.2341	−31.3294
Maximum drawdown	1.5008	1.9924	2.6571	1.7706	2.0731	1.5199	2.3947	2.0619	2.6227	2.693	2.7462	2.6881
Panel C: Annualized risk-return measures												
Annual return	2.5415	0.1912	0.039	0.4961	0.0452	0.2915	2.0449	1.0372	−0.0421	−0.288	−0.0525	−0.2255
Annual sortino	4.7301	0.3558	0.0725	0.9233	0.084	0.5425	3.8058	1.9303	−0.0784	−0.5359	−0.0978	−0.4197

IT.

$$RMSE = \sqrt{MSE}$$

(B.3) accuracy of positive prediction and negative prediction, respectively.

$$H_R = \frac{\text{Count}_{i=1}^n(\hat{r}_i r_i > 0)}{\text{Count}_{i=1}^n(\hat{r}_i r_i \neq 0)}$$

(B.4)

Appendix C. Structure of DL models used in this paper

See Table C1.

$$H_{R+} = \frac{\text{Count}_{i=1}^n(\hat{r}_i > 0 \text{ and } r_i > 0)}{\text{Count}_{i=1}^n(\hat{r}_i > 0)}$$

(B.5)

Appendix E. Results of portfolio optimization models in terms of daily return, daily risk, and annualized risk-return measures for different transaction costs.

See Table E1, Table E2.

$$H_{R-} = \frac{\text{Count}_{i=1}^n(\hat{r}_i < 0 \text{ and } r_i < 0)}{\text{Count}_{i=1}^n(\hat{r}_i < 0)}$$

(B.6)

Appendix F. Daily cumulative return and risk-return ratio of different models for transaction costs of 1% and 2%.

See Fig. F1, Fig. F2, Fig. F3, Fig. F4.

where r_i signifies the actual return while $\hat{r}_i$ denotes the anticipated return. Moreover, H_R denotes total hit rate, H_{R+} and H_{R-} implies the

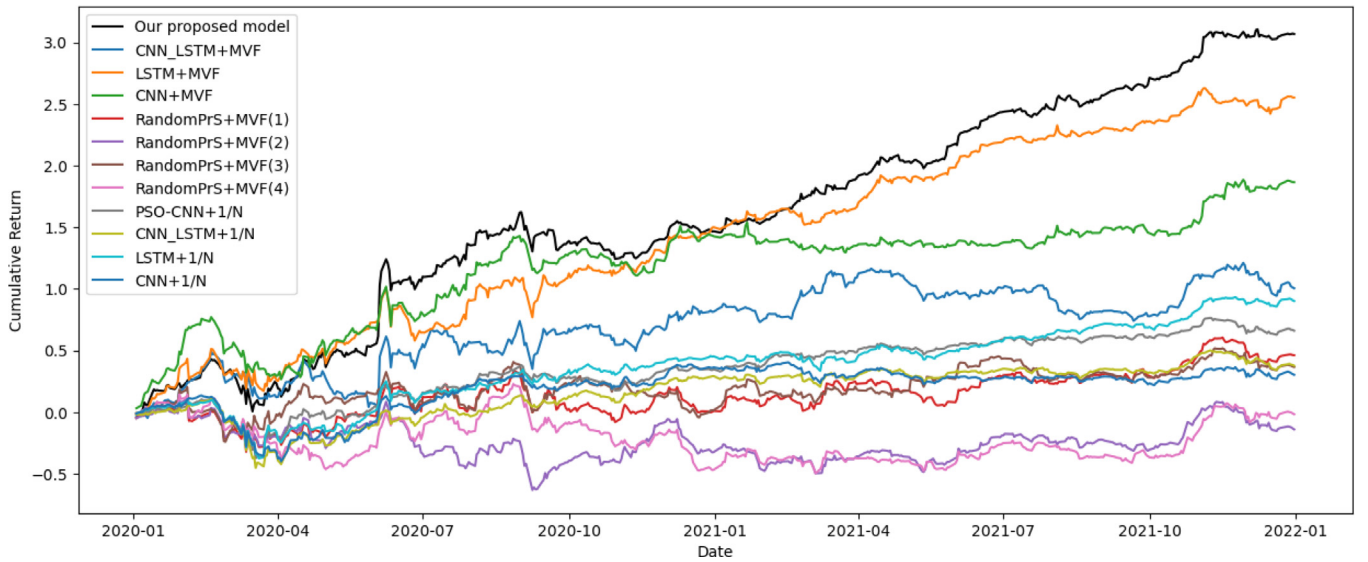


Fig. F1. Cumulative portfolio return with 1% transaction cost.

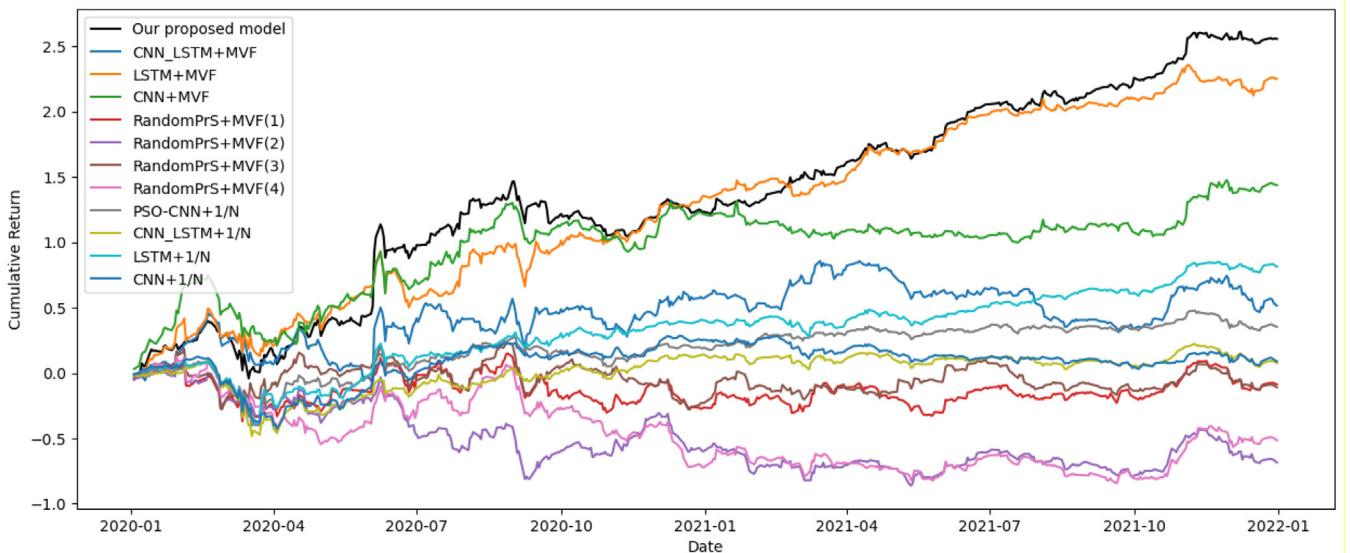


Fig. F2. Cumulative portfolio return with 2% transaction cost.

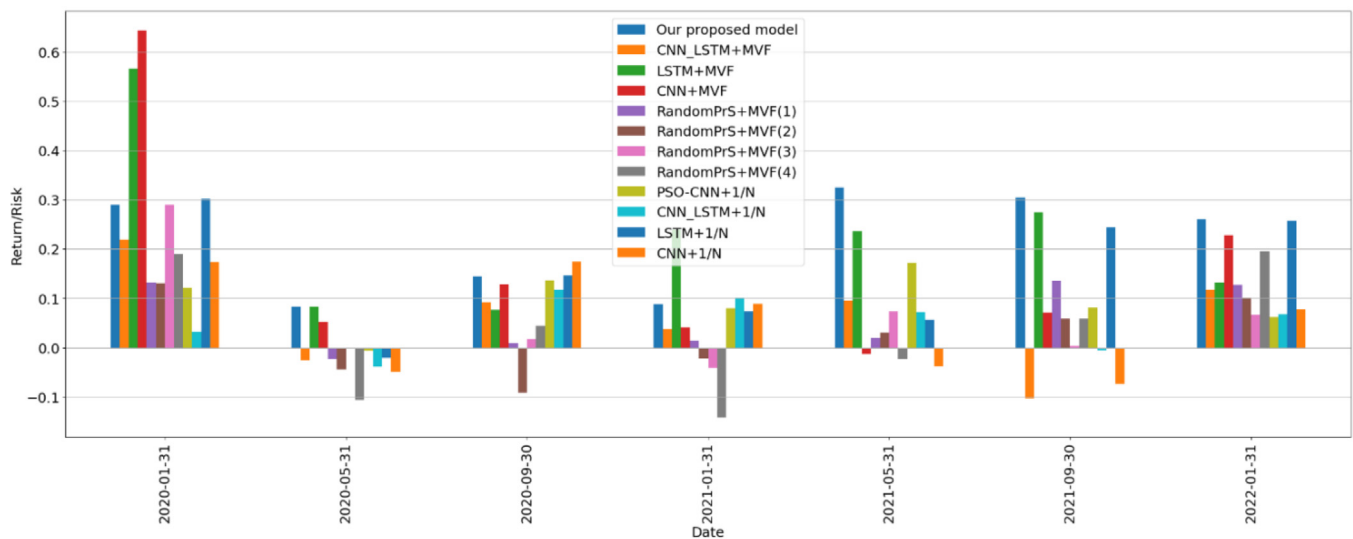


Fig. F3. Return-risk ratio of every two month with 1% transaction costs.

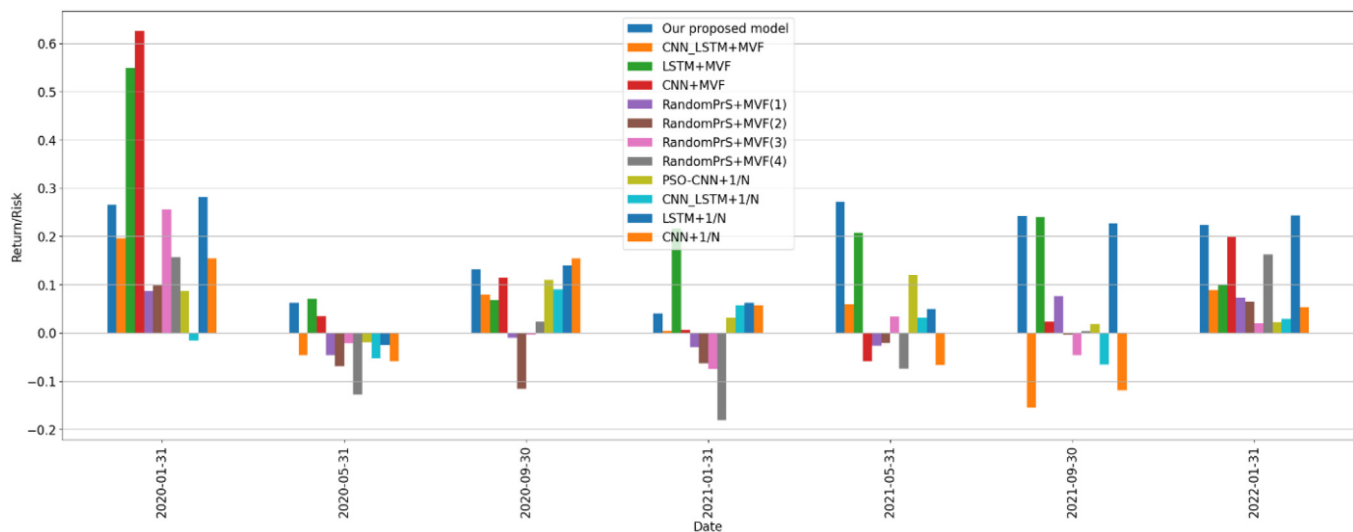


Fig. F4. Return-risk ratio of every two month with 2% transaction costs.

References

- Adebisi, A.A., Adewumi, A.O., Ayo, C.K., 2014. Comparison of ARIMA and artificial neural networks models for stock price prediction. *J. Appl. Math.*
- Alhnaity, B., Abbod, M., 2020. A new hybrid financial time series prediction model. *Eng. Appl. Artif. Intel.* 95 (103), 873.
- Ballings, M., Van den Poel, D., Hespels, N., Gryp, R., 2015. Evaluating multiple classifiers for stock price direction prediction. *Expert Syst. Appl.* 42 (20), 7046–7056.
- Behera, J., Pasayat, A.K., Behera, H., Kumar, P., 2023. Prediction based mean-value-at-risk portfolio optimization using machine learning regression algorithms for multi-national stock markets. *Eng. Appl. Artif. Intel.* 120, 105843.
- Bhandari, H.N., Rimal, B., Pokhrel, N.R., Rimal, R., Dahal, K.R., Khatri, R.K., 2022. Predicting stock market index using LSTM. *Machine Learn. with Appl.* 9, 100320.
- Bini, B.S., Mathew, T., 2016. Clustering and regression techniques for stock prediction. *Procedia Technol.* 24, 1248–1255.
- Bollerslev, T., 1986. Generalized autoregressive conditional heteroskedasticity. *J. Econ.* 31 (3), 307–327.
- Box, G.E., Jenkins, G.M., Reinsel, G.C., Ljung, G.M., 2015. *Time series analysis: forecasting and control*. John Wiley & Sons.
- Cai, J., Luo, J., Wang, S., Yang, S., 2018. Feature selection in machine learning: a new perspective. *Neurocomputing* 300, 70–79.
- Chaweechanchon, A., Chaysiri, R., 2022. Markowitz mean-variance portfolio optimization with predictive stock selection using machine learning. *Int. J. Financial Stud.* 10 (3), 64.
- Chen, W., Zhang, H., Mehlaawat, M.K., Jia, L., 2021. Mean-variance portfolio optimization using machine learning-based stock price prediction. *Appl. Soft Comput.* 100, 106943.
- Chen, W., Zhang, H., Jia, L., 2022. A novel two-stage method for well-diversified portfolio construction based on stock return prediction using machine learning. *North Am. J. Econ. Finance* 63, 101818.
- Engle, R.F., 1982. Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, 987–1007.
- Enke, D., Grauer, M., Mehdiyev, N., 2011. Stock market prediction with multiple regression, fuzzy type-2 clustering and neural networks. *Procedia Comput. Sci.* 6, 201–206.
- Ezzat, D., Hassanien, A.E., Ella, H.A., 2021. An optimized deep learning architecture for the diagnosis of COVID-19 disease based on gravitational search optimization. *Appl. Soft Comput.* 98, 106742.
- Forgy, E., 1965. Cluster analysis of multivariate data: efficiency versus interpretability of classifications. *Biometrics* 21, 768–780.
- Freitas, F.D., De Souza, A.F., De Almeida, A.R., 2009. Prediction-based portfolio optimization model using neural networks. *Neurocomputing* 72 (10–12), 2155–2170.
- Gandhmal, D.P., Kumar, K., 2019. Systematic analysis and review of stock market prediction techniques. *Computer Sci. Rev.* 34, 100190.

- Gunduz, H., Yaslan, Y., Cataltepe, Z., 2017. Intraday prediction of Borsa Istanbul using convolutional neural networks and feature correlations. *Knowl.-Based Syst.* 137, 138–148.
- Gupta, P., 2022. Portfolio optimization using elliptic entropy and semi-entropy of coherent fuzzy numbers. *Inf. Sci.* 614, 240–262.
- Henrique, B.M., Sobreiro, V.A., Kimura, H., 2018. Stock price prediction using support vector regression on daily and up to the minute prices. *J. Finance Data Sci.* 4 (3), 183–201.
- Hochreiter, S., Schmidhuber, J., 1997. Long short-term memory. *Neural Comput.* 9 (8), 1735–1780.
- Hoseinzade, E., Haratizadeh, S., 2019. CNNpred: CNN-based stock market prediction using a diverse set of variables. *Expert Syst. Appl.* 129, 273–285.
- Hua, Z., Xiao, Y., Cao, J., 2021. Misalignment fault prediction of wind turbines based on improved artificial fish swarm algorithm. *Entropy* 23 (6), 692.
- Jensen, M.C., 1978. Some anomalous evidence regarding market efficiency. *J. Financ. Econ.* 6 (2/3), 95–101.
- Jianyu, M., Lingfeng, N., 2016. A survey on feature selection. *Procedia Comput. Sci.* 91, 919–926.
- Kennedy, J., Eberhart, R., 1995. Particle swarm optimization. In *Proceedings of ICNN'95-international conference on neural networks*, Vol. 4, pp. 1942–1948, IEEE.
- Kim, T.Y., Cho, S.B., 2019. Predicting residential energy consumption using CNN-LSTM neural networks. *Energy* 182, 72–81.
- Kodinariya, T.M., Makwana, P.R., 2013. Review on determining number of Cluster in K-Means Clustering. *Int. J.* 1 (6), 90–95.
- Kumar, D.P., 2021. Particle Swarm Optimization: The Foundation. In: *Applying Particle Swarm Optimization: New Solutions and Cases for Optimized Portfolios*, pp. 97–110.
- LeCun, Y., Bottou, L., Bengio, Y., Haffner, P., 1998. Gradient-based learning applied to document recognition. *Proc. IEEE* 86 (11), 2278–2324.
- Li, E., Zhou, J., Shi, X., Jahed Armaghani, D., Yu, Z., Chen, X., Huang, P., 2021. Developing a hybrid model of salp swarm algorithm-based support vector machine to predict the strength of fiber-reinforced cemented paste backfill. *Eng. Comput.* 37, 3519–3540.
- Lin, C.Y., 2021. Forward stepwise random forest analysis for experimental designs. *J. Qual. Technol.* 53 (5), 488–504.
- Livieris, I.E., Pintelas, E., Pintelas, P., 2020. A CNN-LSTM model for gold price time-series forecasting. *Neural Comput. & Applic.* 32, 17351–17360.
- Lloyd, S., 1982. Least squares quantization in PCM. *IEEE Trans. Inf. Theory* 28 (2), 129–137.
- Lorenzo, P.R.; Nalepa, J.; Kawulok, M.; Ramos, L. S.; Pastor, J.R., 2017. Particle swarm optimization for hyper-parameter selection in deep neural networks. In: *Proceedings of the genetic and evolutionary computation conference*, pp. 481–488.
- Ma, Y., Han, R., Wang, W., 2021. Portfolio optimization with return prediction using deep learning and machine learning. *Expert Syst. Appl.* 165, 113973.
- Ma, Y., Wang, W., Ma, Q., 2023. A novel prediction based portfolio optimization model using deep learning. *Comput. Ind. Eng.* 109023.
- MacQueen, J., 1967. Some methods for classification and analysis of multivariate observations.
- Markowitz, H.M., 1952. Portfolio selection. *J. Financ.* 7 (1), 77–91.
- Nayak, J., Naik, B., Dash, P.B., Sour, A., Shanmuganathan, V., 2021. Hyper-parameter tuned light gradient boosting machine using memetic firefly algorithm for hand gesture recognition. *Appl. Soft Comput.* 107, 107478.
- Nobre, J., Neves, R.F., 2019. Combining principal component analysis, discrete wavelet transform and XGBoost to trade in the financial markets. *Expert Syst. Appl.* 125, 181–194.
- Paiva, F.D., Cardoso, R.T.N., Hanaoka, G.P., Duarte, W.M., 2019. Decision-making for financial trading: A fusion approach of machine learning and portfolio selection. *Expert Syst. Appl.* 115, 635–655.
- Rasel, R.I., Sultana, N., Meesad, P., 2015. An efficient modelling approach for forecasting financial time series data using support vector regression and windowing operators. *Int. J. Comput. Intell. Stud.* 4 (2), 134–150.
- Roondiwala, M., Patel, H., Varma, S., 2017. Predicting stock prices using LSTM. *Int. J. Sci. Res. (IJSR)* 6 (4), 1754–1756.
- Sezer, O.B., Ozbayoglu, A.M., 2018. Algorithmic financial trading with deep convolutional neural networks: Time series to image conversion approach. *Appl. Soft Comput.* 70, 525–538.
- Ta, V.D., Liu, C.M., Tadesse, D.A., 2020. Portfolio optimization-based stock prediction using long-short term memory network in quantitative trading. *Appl. Sci.* 10 (2), 437.
- Wang, Y., Aste, T., 2023. Dynamic portfolio optimization with inverse covariance clustering. *Expert Syst. Appl.* 213, 118739.
- Wang, W., Li, W., Zhang, N., Liu, K., 2020. Portfolio formation with pre-selection using deep learning from long-term financial data. *Expert Syst. Appl.* 143, 113042.
- Wang, Y., Zhang, H., Zhang, G., 2019. cPSO-CNN: An efficient PSO-based algorithm for fine-tuning hyper-parameters of convolutional neural networks. *Swarm Evol. Comput.* 49, 114–123.
- Wicaksono, A.S., Supianto, A.A., 2018. Hyper parameter optimization using genetic algorithm on machine learning methods for online news popularity prediction. *Int. J. Adv. Comput. Sci. Appl.* 9 (12).
- Yang, L., Shami, A., 2020. On hyperparameter optimization of machine learning algorithms: theory and practice. *Neurocomputing* 415, 295–316.
- Yu, J.R., Chiou, W.J.P., Lee, W.Y., Lin, S.J., 2020. Portfolio models with return forecasting and transaction costs. *Int. Rev. Econ. Financ.* 66, 118–130.
- Zhang, X.D., Li, A., Pan, R., 2016. Stock trend prediction based on a new status box method and AdaBoost probabilistic support vector machine. *Appl. Soft Comput.* 49, 385–398.
- Zhou, F., Zhang, Q., Sornette, D., Jiang, L., 2019. Cascading logistic regression onto gradient boosted decision trees for forecasting and trading stock indices. *Appl. Soft Comput.* 84, 105747.
- Zolfani, S.H., Taheri, H.M., Gharehgozlou, M., Farahani, A., 2022. An asymmetric PROMETHEE II for cryptocurrency portfolio allocation based on return prediction. *Appl. Soft Comput.* 131, 109829.